

Gian Maria Troiani

Knologi Internship

Augmenting Large Language Models with Knowledge Graphs

Index:

Part I: The Problem Domain

Part II: Case Study - City of Coral Gables

Part III: Architectural Experimentation

Part IV: Implementation Framework

Part V: The Unified-Graph Municipal Knowledge System

PROMPTS

ONTOLOGY

Part I: The Problem Domain

Introduction

What does it mean for an AI system to truly understand and work with documents? This fundamental question underlies thousands of enterprise deployments, from municipal governments processing vast collections of ordinances and resolutions to corporations managing complex webs of contracts and policies. Intelligent document use requires four essential capabilities: providing accurate answers with clear provenance, achieving complete retrieval across defined document collections, maintaining updates that reflect the current state of records, and preserving the contextual understanding that makes information actionable. Context encompasses the organizational knowledge, domain expertise, specialized terminology, temporal factors, and procedural history that give documents their meaning. Much of this critical context exists outside the documents themselves, residing in institutional memory and professional practice.

Organizations invest in LLMs hoping to tame their expanding document repositories, only to watch promising pilots fail to deliver production-grade results. LLMs excel at initial exploration tasks: identifying relevant passages, generating summaries, and connecting dispersed information into coherent narratives. Yet as document collections expand and accuracy standards rise, their fundamental probabilistic nature becomes a liability. When corpus content exhibits high similarity, which is common in legal documents, technical specifications, and corporate policies, retrieval precision degrades as vector embeddings struggle to distinguish between semantically similar but contextually distinct documents. They cannot guarantee reliable, complete, or current outputs. Configuration options affect generation style and retrieval scope but cannot enforce hard guarantees or maintain service-level agreements. This is not a problem that better prompting or fine-tuning can solve. It represents a fundamental architectural incompatibility between intelligent document processing requirements and transformer model capabilities. True document intelligence demands deterministic retrieval, exhaustive corpus coverage, persistent organizational knowledge, and verified contextual grounding. These are guarantees that LLM architecture cannot make. Bridging this gap requires a hybrid architecture that wraps the LLM's probabilistic reasoning in deterministic infrastructure, anchoring every output to verifiable sources of truth.

This paper will argue that by augmenting the linguistic fluency and "reasoning power" of LLMs with a structured, verifiable, and explicit architectural complement, it is possible to create powerful AI systems that achieve the accuracy, completeness, and grounded context required for increased and more reliable document intelligence. The paper will also outline the blueprint of such an application.

Beyond document intelligence, building these hybrid systems creates a foundation of structured, queryable knowledge that extends far beyond chat interfaces. The same verified, complete, and continuously updated knowledge graph that enables reliable document querying can power downstream applications: automated workflows that trigger based on extracted relationships, machine learning models that leverage graph embeddings and structured features, agentic systems that autonomously execute complex tasks using verified knowledge as their operational context, and decision support tools that combine structured reasoning with natural language understanding. By investing in this architectural approach, organizations build not just a better search system, but a comprehensive knowledge infrastructure that can serve as the backbone for intelligent automation across multiple domains.

However, achieving this vision requires first understanding precisely why LLMs alone, despite their remarkable capabilities, cannot deliver the guarantees these systems demand. Their limitations are not bugs to be patched or features to be added, but fundamental consequences of their probabilistic architecture. The following section examines these architectural constraints in detail, demonstrating why external structure is not optional enhancement but essential scaffolding for mission-critical applications.

Large Language Models represent a breakthrough in artificial intelligence, trained on vast text corpora to predict and generate human language with remarkable fluency. These transformer-based systems excel at tasks once thought exclusive to human intelligence: answering complex questions, summarizing lengthy documents, translating between languages, generating coherent long-form text, and even exhibiting forms of reasoning through chain-of-thought processes. Their ability to understand context, follow instructions, and adapt to diverse tasks through prompting alone has revolutionized how we interact with information. LLMs can process natural language queries without rigid syntax requirements, extract insights from unstructured text, and generate responses that often match or exceed human quality in many domains. This versatility and accessibility explain their rapid adoption across industries, from customer service and content creation to code generation and research assistance. Yet precisely because these models appear so capable, understanding their fundamental architectural constraints becomes crucial for deploying them responsibly in systems where accuracy, completeness, and reliability are non-negotiable.

Critical Architectural Limitations of LLMs

LLMs operating in isolation cannot deliver the guarantees required for intelligent document processing. These fundamental architectural constraints explain why. Appendix A provides additional technical limitations.

1) Fundamental Challenge: Probabilistic Generation

LLMs produce outputs through statistical token selection from probability distributions rather than deterministic computation. This probabilistic architecture makes confabulation unavoidable without external validation. When facing uncertainty, models generate statistically plausible tokens that often create convincing yet false information. Even advanced models designed for reasoning tasks still operate through probability sampling. Without external verification mechanisms or hard constraints, these systems cannot provide the deterministic guarantees that mission-critical applications demand.

2) Structural Blindness: Inability to Guarantee Completeness

LLMs interpret relationships as scattered textual patterns rather than maintaining explicit inventories of existing entities. They cannot guarantee exhaustive enumeration when tasked with listing all instances of a particular type. The model has no mechanism to determine whether it has identified 10 percent or 100 percent of relevant items. Standard top-k retrieval augmentation permanently discards results below threshold cutoffs. While iterative retrieval and recall-focused approaches can minimize omissions, true completeness guarantees require an external index that formally defines the corpus boundaries and enables systematic coverage verification. When asked to identify all regulations affecting downtown development, completeness depends on a properly governed corpus definition and purposefully designed indices, not on clever prompting strategies.

3) Frozen Knowledge: Absence of Persistent Organizational Learning

Model parameters remain frozen following training. Fine-tuning merely produces another static snapshot. Production deployments cannot update their weights through regular user interactions. Lasting organizational adaptation necessitates external memory systems, version-controlled storage, and proper governance frameworks. While prompt engineering can inject temporary context, this information disappears after each session, requiring users to repeatedly specify terminology and mappings. A system that processes the abbreviation "app ref #" thousands of times never permanently learns it means "application reference number" without external storage and governance mechanisms.

4) Metacognitive Absence: Cannot Reliably Identify Information Needs or Abstain

LLMs possess no inherent mechanisms to identify what they do not know, determine which documents could resolve specific uncertainties, or recognize when to withhold responses. While they generate probability scores, these are poorly calibrated and unreliable indicators of factual accuracy. Fluent output provides no signal about whether adequate evidence was considered. Without supplementary systems, models display identical confidence regardless of whether their information is comprehensive or entirely absent.

What engineering workarounds help, and where they stop

Various techniques can enhance reliability for specific use cases: constrained decoding, tool integration, expanded retrieval windows, iterative search, ensemble voting, and automated verification systems. While these approaches reduce error rates, they cannot provide the absolute guarantees critical operations require. At the time of this writing, language modeling alone without external structure, validation, and memory, cannot deliver deterministic accuracy, comprehensive retrieval, continuous organizational learning, or properly calibrated uncertainty. Complementary technologies like relational databases, knowledge graphs, adaptive learning systems, and symbolic reasoners can address these specific gaps, though they lack the natural language capabilities of LLMs. The central insight remains that LLMs, while extraordinarily capable for many applications, should not serve as the exclusive foundation for systems demanding guaranteed correctness, completeness, currency, or organizational adaptation.

Understanding Knowledge Graphs

Knowledge graphs represent a fundamentally different approach to information storage and retrieval. Unlike the probabilistic token generation of LLMs, knowledge graphs store information as explicit nodes (entities) and edges (relationships) in a deterministic structure. This property graph model maintains concrete catalogs of what exists in the system, with each entity containing properties stored as key-value pairs and relationships that explicitly connect entities with typed, directional links. Knowledge graphs support organizing principles like taxonomies for hierarchical classification and ontologies for complex networked relationships, enabling both "X is a kind of Y" and "X is related to Y" reasoning.

Critically, they offer what LLMs structurally cannot: deterministic querying where traversal time scales with the subgraph visited rather than total database size, complete enumeration guarantees through explicit indexing, dynamic updates that persist organizational knowledge, and the ability to enforce constraints and rules that govern data consistency. Modern graph databases scale to billions of connections while maintaining these guarantees. Their power lies in layered intelligence: a municipal knowledge graph might connect a property address to its zoning classification, link that to applicable ordinances, connect those ordinances to the

commission meetings where they were approved, and associate those meetings with the stakeholders who testified. A single query can traverse these layers to answer "What restrictions apply to this property and who approved them?", returning not just the answer but the complete provenance chain.

However, knowledge graphs come with significant trade-offs. They require explicit modeling of entities and relationships, often demanding manual ontology design and ongoing maintenance as domains evolve. Entity resolution and deduplication require constant attention to prevent the same real-world object from fragmenting into multiple nodes. Schema evolution can be complex, requiring careful migration strategies. Unlike LLMs that can immediately process any text, knowledge graphs need structured extraction pipelines to transform unstructured documents into nodes and edges. This upfront investment in structure and ongoing maintenance creates a higher barrier to entry than LLMs, requiring organizations to thoughtfully plan their implementation strategy to fully realize the benefits of deterministic, auditable, and complete information retrieval.

Bridging the Gap: Knowledge Graphs as Architectural Complement

While LLMs cannot transcend their architectural constraints, combining them with knowledge graphs creates a hybrid system that addresses the critical gaps. Knowledge graphs provide what chat-style systems lack: deterministic retrieval, explicit structure, relationship completeness, dynamic updates, and governed context. Where LLMs treat text as undifferentiated signals, knowledge graphs maintain explicit entities, relationships, and hierarchies with queryable certainty. They can answer corpus-complete questions such as "show me all regulations affecting downtown development" provided the pipeline has fully ingested and normalized the corpus under governed taxonomies and ontologies. They can update organizational knowledge in real time, capture that "app ref #" means "application reference number" once, and retain it permanently with provenance. They also enable calibrated responses by supporting auditable completeness checks and explicit abstention when coverage is insufficient.

The synergy is powerful. LLMs handle natural language understanding, context interpretation, and fluent communication. Knowledge graphs provide the structured backbone for factual accuracy, complete enumeration, persistent organizational memory, and grounded context. This is not a replacement for LLMs but an augmentation with complementary architecture. The LLM interprets user intent and assembles human-friendly responses. The knowledge graph ensures correctness, completeness, freshness, and organizational alignment.

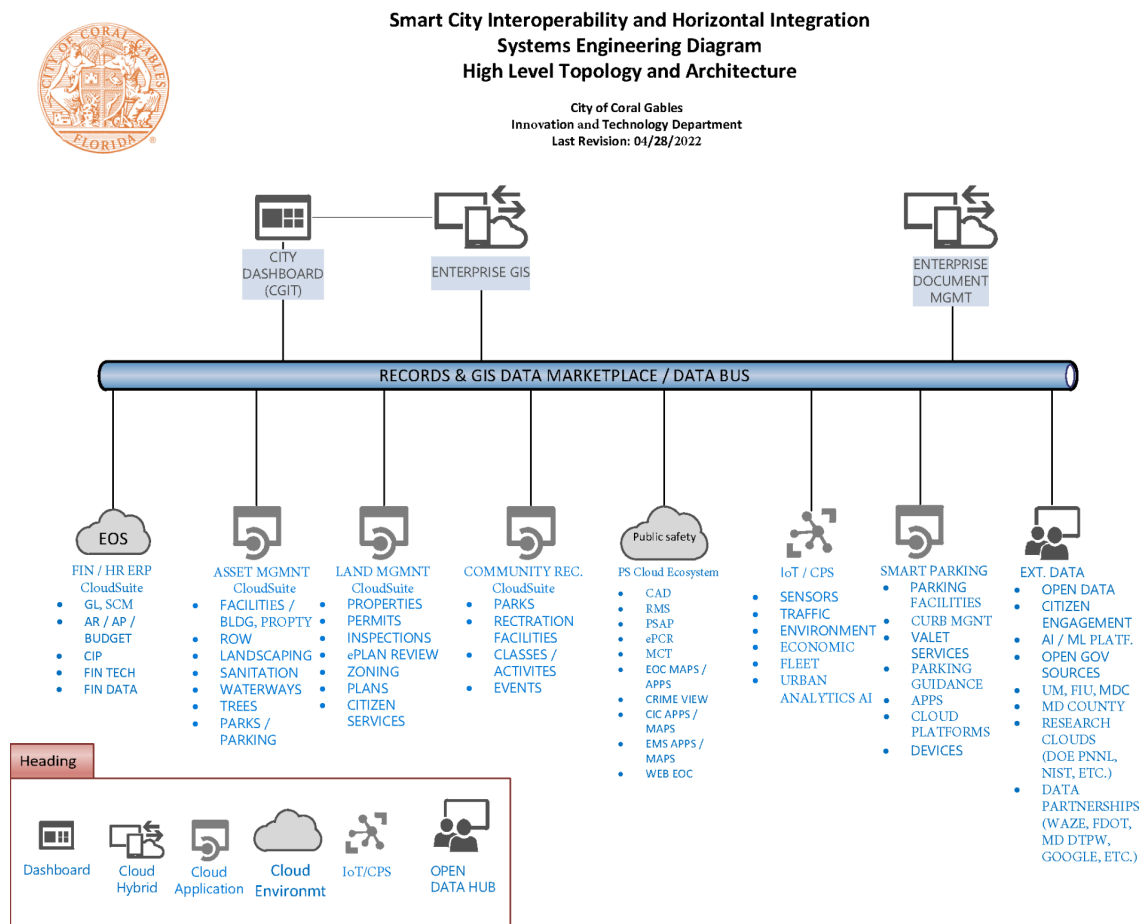
These theoretical considerations take on practical urgency in real-world deployments where organizations must process thousands of technical documents with legal requirements for completeness and accuracy. The City of Coral Gables provides an instructive case study of how these architectural challenges manifest in municipal document management, where the stakes of incomplete retrieval or inaccurate information directly impact civic governance and public service.

Part II: Case Study - City of Coral Gables

Municipal Context and Vision: Coral Gables General Problem Statement

The massive transformative purpose at the Innovation and technology department IoT and GIS lab is to "know everything in and around the city". Coral Gables plans to achieve this objective, the data needs to abide by the FAIR standard (findability, accessibility, interoperability, and reusability).

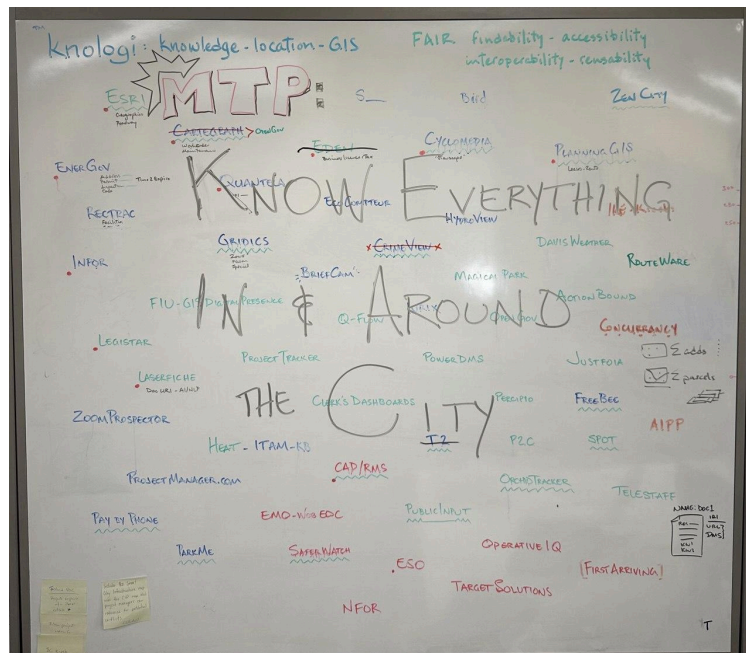
The path to arrive at such a vision started years ago with data capture in many different areas of the city's life. Some examples include data about geography (maps), data about the various departments (fire, police, etc.) and citizens quality of life (air quality, traffic data, parks data), etc. While collecting data is a necessary step to provide data-driven intelligence, the data needs to be consumed by applications to be interacted with, provide analytics and be reasoned over. The horizontal integration and Smart City Interoperability and Horizontal Integration diagram graphically represents this vision (figure below). The diagram shows the intent to know what is in every system, what kind of data it contains, and exposing the data to be able to use it. The aim is to use that data in creative ways, use different datasets in different ways and make the system overall "intelligent".



Knowledge graphs are a powerful tool to organize real-world entities, their relationships and attributes. Their relationships capture the meaning and context of how they are related. For this reason, they are a flexible

tool to represent data in a rich and customized way that makes it especially accessible for humans. Because of their useful properties, the IoT and GIS lab is looking to represent all the data in their systems with knowledge graphs.

Below you can see a picture of one of the whiteboards in the IoT and GIS lab. The names on the boards represent various data sources and internal systems that the lab wants to represent digitally to "know everything in and around the city", ideally leveraging the power of knowledge graphs. These overall sub-systems are part of a bigger data ecosystem of the city.



The City Clerk Challenge

This internship project tackled these challenges by building an application to process City Clerk documents. The project tested several approaches and architectures to experiment with viable solutions and test hypotheses, eventually settling on a Hybrid RAG system. The ideal solution needed to process and enable intelligent querying over several years of city meeting documents. These documents are generated every two weeks when the city commission meets, with archives extending back 90 years. The corpus contains thousands of long-form, official bureaucratic and technical documents. Each meeting produces several document types including Agendas, Ordinances, Resolutions and Verbatims. These documents capture key aspects discussed, approved, or rejected during city commission meetings, such as city projects, development plans, laws passed, and municipal decisions in Coral Gables. The Hybrid RAG needs to answer questions from the City Clerk about these documents both intelligently and reliably. Eventually, the same application will extend to residents, allowing them to inquire about commission meeting discussions and decisions.

System Requirements and Constraints

The system required a chat-like interface for easy interaction with the information. Critically, a good user experience took priority over the underlying technology. Users needed to ask questions about municipal documents and receive well-informed answers. The conversation had to be seamless, minimizing cognitive load while ensuring users received sufficient information to fully answer their queries. An early iteration of the

system is shown below.

The City Clerk application serves as a proving ground for the larger vision of comprehensive city knowledge management. The architecture developed here, designed to process decades of commission documents with guaranteed completeness, would demonstrate the viability of extending this approach to the city's entire data ecosystem.

Meeting these requirements demanded systematic exploration of different architectural approaches. The following section details the experimental journey from simple taxonomies to sophisticated hybrid systems, revealing critical insights about what works, and what doesn't, when designing systems to process municipal documents at scale.

To provide more context, the next pages will provide some samples of City Clerk documents to be extracted:

Ordinance example:

City of Coral Gables

405 Biltmore Way
Coral Gables, FL 33134
www.coralgables.com



Agenda

Tuesday, January 14, 2014

9:00 AM

City Hall, Commission Chambers

City Commission

Mayor Jim Cason
Vice Mayor William H. Kerdyk, Jr.
Commissioner Patricia Keon
Commissioner Vince Lago
Commissioner Frank C. Quesada

Patrick G. Salerno, City Manager
Craig E. Leen, City Attorney
Walter J. Foeman, City Clerk

Ordinance example:

Page 3 of 3 – Ordinance No. 2024-02

Ordinance example:

CITY OF CORAL GABLES, FLORIDA

RESOLUTION NO. 2014-17

RESOLUTION AUTHORIZING SAINT GEORGE ANTIOCHIAN ORTHODOX CATHEDRAL TO SELL ALCOHOLIC BEVERAGES AS PART OF THE ANNUAL CATHEDRAL FESTIVAL ON 320 PALERMO AVENUE ON SATURDAY, FEBRUARY 8, 2014 FROM 12:00 P.M. TO 11:00 P.M. AND ON SUNDAY, FEBRUARY 9, 2014 FROM 12:00 P.M. TO 3:00 P.M., SUBJECT TO FLORIDA DEPARTMENT OF PROFESSIONAL REGULATION REQUIREMENTS.

BE IT RESOLVED BY THE COMMISSION OF THE CITY OF CORAL GABLES:

SECTION 1. That permission is hereby granted to Saint George Antiochian Orthodox Cathedral to sell alcoholic beverages as part of the Annual Cathedral Festival on 320 Palermo Avenue on Saturday, February 8, 2014 from 12:00 p.m. to 11:00 p.m. and on Sunday, February 9, 2014 from 12:00 p.m. to 3:00 p.m., subject to Florida Department of Professional Regulation Requirements.

SECTION 2. That this resolution shall become effective upon the date of its adoption herein.

PASSED AND ADOPTED THIS TWENTY-EIGHT DAY OF JANUARY, A.D., 2014.

(Moved: Quesada / Seconded: Kerdyk)
(Yeas: Kerdyk, Lago, Quesada, Keon, Cason)
(Unanimous: 5-0 Vote)
(Agenda item: C-1)

APPROVED:


JIM CASON
MAYOR

ATTEST:


WALTER J. FOEMAN
CITY CLERK

APPROVED AS TO FORM
AND LEGAL SUFFICIENCY:


CRAIG E. LEEN
CITY ATTORNEY

Monthly BOD	Multiplier
250 ppm or less	1.000
260	1.025
270	1.050
280	1.075
290	1.100
300	1.125
400	1.375
500	1.625
10000	2.875

16. To provide for and bear the cost of sampling with suitable sampling facilities, when reasonable cause for sampling exists. The City shall give the customer or tenant reasonable notice when sampling is necessary, and qualified City representatives shall thereafter perform the necessary sampling as efficient as possible.

17. To connect to the City sewer systems at the customer's expense in a manner acceptable to the City, when sewerage is completed to a new area in the City which can more efficiently and effectively serve the customer's outside connection.

18. To provide liability insurance in the amounts required by Resolution No. 22601, naming the City of Coral Gables as additional insured, and covering any damages to public or private property due to failure in the customer's facilities. A certificate of insurance shall be required at the execution of the agreement in a form acceptable to the City of Coral Gables.

19. To provide a maintenance bond or other surety in the amount of five (5) percent of the construction cost to assure timely repair of the customer's facilities should a failure occur, said surety to run in perpetuity or until the connection is no longer required.

20. To bear expense of recording the agreement encompassing the above terms in the Public Records of Dade County, Florida, and said agreement shall be a covenant running with the land which will state that the owner will not convey or cause to be conveyed, the title to the above property without requiring the successor in title to abide by all the terms and conditions of said agreement.



KEITH and SCHNARS, P.A.
ENGINEERS, PLANNERS, SURVEYORS

City of Coral Gables Public Works
University of Miami, UHealth Gables Medical Facility
December 22, 2014 – Page 4

Verbatim example:

City of Coral Gables City Commission Meeting Agenda Item F-1 February 11, 2014 City Commission Chambers 405 Biltmore Way, Coral Gables, FL

City Commission

Mayor Jim Cason
Vice Mayor William H. Kerdyk, Jr.
Commissioner Pat Keon
Commissioner Vince Lago
Commissioner Frank Quesada

City Staff

City Manager, Patrick Salerno
City Attorney, Craig E. Leen
City Clerk, Walter J. Foeman
Deputy City Clerk, Billy Urquia

Public Speaker(s)

Leona Cooper, Coral Gables Resident
Daisy Fuller, Coral Gables Resident
Roxey Bolton and Daughter, Coral Gables Residents
Debra Kolsky, President of DEVCO
Judith Davis, Coral Gables Resident

F-1 [Start: 10:04:30 a.m.]

Discussion regarding extension of trolley service.
Commissioner Vince Lago

Mayor Cason: The next item, F-1 is a discussion of the extension of the trolley service, Commissioner Lago, this is your item.

Commissioner Lago: Yes sir. Mayor thank you. Craig, if you'll be so kind I'd like to read the resolution. Thank you sir.

Commissioner Keon: You want to speak?

City Commission Meeting
February 11, 2014
Agenda Item F-1

route, different areas. You imagine being able to connect in the City of Miami, something I'm going to bring up to them, if we can connect at whatever bus stop we have. Excuse me, trolley stop, I apologize, and then move forward someone can hop on, on a City of Miami trolley and ride it all the way to Brickell, that would be spectacular, that's the key. From my understanding, again Mr. Manager, we were discussing funding in regards to this project, this is a fully funded project from the County, this does not come from City of Coral Gables dollars, am I correct? or am I wrong?

City Manager Salerno: No.

Commissioner Lago: No? What is it?

City Manager Salerno: All of the funds are City funds, they come from the...

Commissioner Quesada: People's Transportation Plan.

City Manager Salerno: Right -- and they have to be used for transportation related projects, could be traffic improvements, etc., but right now those funds are all committed toward transportation.

Commissioner Lago: So it comes from the half penny tax.

Commissioner Quesada: Hold on a second. I think we get about \$1.3 million from the People's Transportation Plan and then we put an additional \$300,000, if I recall the budget correctly, is that right?

City Manager Salerno: No.

Mayor Cason: \$600,000 -- something?

City Manager Salerno: Right now we live within -- Commissioner you are correct, the \$1.3 million, that is the amount that we operate within today.

Commissioner Lago: Where does that \$1.3 million come from?

Commissioner Quesada: The half penny sales tax increase.

City Manager Salerno: As Commissioner Quesada, the half-penny sales tax.

City Commission Meeting
February 11, 2014
Agenda Item F-1

Part III: Architectural Experimentation

Experimental Journey: From Simple Taxonomies to Hybrid Systems

The project explored several query methodologies, each with distinct capabilities and limitations:

The screenshot shows the 'City Clerk GraphRAG Query System' interface. At the top, there's a header with a city hall icon and the title. Below it is a search bar labeled 'Enter Your Query'. Inside the search bar, there's a placeholder text: 'Ask about agenda items, ordinances, resolutions, or city proceedings...'. Below the search bar, there are 'Examples:' listed: '- What is agenda item E-1?', '- Tell me about ordinance 2024-01', and '- What are the main development themes?'. To the left of the search bar, there's a 'Query Method:' section with five radio button options: 'Auto-Select (Recommended)', 'Agent Graph Query (Advanced)', 'Local Search', 'Global Search', and 'DRIFT Search'. Below these is 'Simple NER (No GraphRAG)'. To the right of the search bar, there's a 'Query Options:' section with four checkboxes: 'Include community context' (checked), 'Show routing details' (checked), 'Show data sources' (checked), and 'Verbose results' (unchecked). At the bottom of the interface, there are two buttons: a blue 'Submit Query' button and a grey 'Clear' button.

- **Simple knowledge graph taxonomy** built relationships between documents in the City Clerk office. This approach excelled at navigating document hierarchies, tracking document lineage, and handling cross-references within single commission meetings. The taxonomy effectively captured relationships between documents produced in the same meeting. However, it fell short in several areas: it could not handle relationships between documents across different meeting dates, failed to capture non-hierarchical relationships, and missed document semantics entirely. These limitations led to exploring more sophisticated approaches.
- **Knowledge graph retrieval based on Named Entity Recognition (NER)** used natural language processing to automatically identify and categorize named entities within unstructured text, such as people names, departments, and addresses. This approach extracted triplets (subject-predicate-object relationships) from source documents to build a graph of entities, relationships, and attributes. The experimentation explored multiple implementation strategies:
 - Extracting triplets without an ontology first and classifying entities, relationships and attributes later (an ontology is essentially a graph database self-enforced schema)
 - Extracting triplets with an ontology to guide the extraction process
 - Extracting triplets using various levels of ontology complexity to understand how it affects extraction times and extraction accuracy

- Extracting triplets (entities, relationships, attributes) with an ontology to guide the extraction process in one pass (providing the full ontology in the context window of an LLM)
- Extracting with an ontology to guide the extraction process but isolating the extraction of entities, relationships, and attributes respectively, and linking them afterwards
- Extracting triplets (entities, relationships, attributes) with an ontology to guide the extraction process in several passes (providing only a part of the ontology in the context window of an LLM)

This systematic NER exploration proved fundamental for understanding how the system responded to various configurations and identifying the most effective approaches.

- **Microsoft GraphRAG** offered an automated framework that builds entire knowledge graphs from raw source documents by extracting entities and relationships, then applying the Leiden algorithm for hierarchical community detection. The Leiden algorithm automatically identifies clusters of related entities and organizes them into communities at multiple levels, enabling the system to understand both local neighborhoods and global themes without manual taxonomy design. While this automated graph construction proved powerful and delivered good overall results in testing, it could not provide exhaustive enumerations regardless of query type, which was essential for the specific system requirements. Additionally, the approach required complete re-processing every time documents updated to reorganize communities, making it extremely expensive in resources, time, and money. Given that municipal documents update every two weeks and require days of processing for the full corpus, this approach became unfeasible. The system also sacrificed granular control over graph construction, offered limited visibility into community formation, and prevented fixing misclassified entities without complete re-processing:

The method provides three search approaches:

- **Local search** retrieved information by starting from specific entities and traversing their immediate graph neighborhoods, making it ideal for targeted questions about particular departments, people, or addresses. It excelled at specific queries about permits for particular addresses and provided strong provenance tracking, though it remained limited to entity-specific context and could not answer broad policy questions while requiring entity embeddings storage.
- **Global search** used pre-computed community summaries generated from the Leiden algorithm's hierarchy to answer broad thematic questions, excellently identifying patterns across all documents and perfect for queries about development policy trends. While it captured themes across documents at multiple abstraction levels, it incurred significantly higher token costs and relied on static summaries that became stale between biweekly updates.
- **DRIFT search** dynamically combined both approaches by starting with community-level context from global search, then iteratively refining using local search traversals. This provided the most comprehensive answers and significantly outperformed local search alone while handling both specific and broad queries, though it proved computationally expensive, complex to debug, took

several minutes per answer, and was overkill for simple lookups.

- **RAG only** applications based solely on vector databases for document retrieval were tested in two flavors:
 - **RAG** using vector similarity (cosine similarity) provided quick setup, handled conceptual queries well, and managed paraphrasing effectively. However, it demonstrated relatively low accuracy on legal text, struggled with exact alphanumeric queries like document numbers or case references, and missed important results when corpus content had high semantic similarity. It also could not guarantee exhaustive enumeration or complete retrieval, critical requirements for compliance and legal queries.
 - **RAG with keyword search** enhanced the standard approach by infusing ranking scores with classical deterministic keyword search to compensate for potentially excluded important results from semantic-only systems. This hybrid significantly improved accuracy on municipal documents by catching exact terms, acronyms, and document numbers that pure semantic search often missed, providing the best balance for mixed query types. However, it required maintaining and tuning two separate systems, increased processing time, and added implementation complexity while demanding careful score fusion and component synchronization.

Convergence on Hybrid Architecture

The final approaches combined both vector databases and knowledge graphs with a chat interface, recognizing that different query types required different retrieval strategies. Simple keyword searches worked best with deterministic matching, semantic questions benefited from vector similarity, relationship queries needed graph traversal, and exhaustive enumeration required the explicit cataloging that knowledge graphs provide. Since no single approach handled all cases well, hybrid architectures became necessary.

The first hybrid approach followed a sequential workflow:

- **KG retrieval with RAG and keyword search:** When users submitted queries, the system sent them to an LLM that generated Cypher queries based on the graph database schema and user intent. After returning knowledge graph results, the system passed them downstream to a RAG and keyword search component featuring dynamically adjustable top-k retrieval parameters. This adjustment provided appropriate context to knowledge graph hits regardless of result count. The top-k parameter determines how many vector database results to retrieve. All gathered information then flowed to LLMs that synthesized the data and generated user answers.

Subsequent approaches experimented with less sequential, more complex flows where queries underwent intent classification to route them to appropriate backends. This classification determined whether queries needed semantic understanding from vector databases, structural relationships and complete enumeration from knowledge graphs, or both. For instance, "What did the mayor say about parking?" routed to vectors for semantic search, while "List all ordinances affecting Main Street properties" routed to the graph for exhaustive enumeration. Both approaches maintained dynamically adjusting top-k parameters that adapted based on knowledge graph query results:

- **LLM-based agent with KG retrieval and RAG** employed an autonomous agent with access to multiple tools including vector search, graph queries, and keyword search, dynamically deciding which tools to invoke based on queries. The agent could chain multiple retrievals, refine searches based on initial results, and combine outputs from different sources. While this approach successfully handled complex multi-hop queries and demonstrated impressive flexibility, it was ultimately set aside to provide an application with more predictable, auditable decision paths.
- **A stricter LLM-based workflow** emerged after exploring the agent approach, implementing a more deterministic workflow where an LLM classified queries into predefined categories, then followed fixed execution paths for each category. This design choice limited system moving parts for higher reliability while prioritizing consistency and auditability. The workflow delivered predictable performance, easier debugging, and clearer decision traceability.

While these experiments revealed the optimal technical architecture, implementing such a system requires careful attention to knowledge modeling. Before examining the final system blueprint, it's essential to understand how to construct the ontologies that form the semantic backbone of any knowledge graph implementation.

Part IV: Implementation Framework

Creating Domain Ontologies

Creating an ontology:

In the context of knowledge management and information systems, an ontology is a formal representation of knowledge within a domain. It consists of three main elements:

1. entities (the objects or concepts that exist in your domain)
2. relationships (how these entities connect to and interact with each other)
3. attributes (the properties or characteristics that describe each entity).

Together, these elements create a structured framework that allows both humans and machines to understand and reason about the information in your application. Building ontologies is a topic that could fill entire books, and indeed has, with debates about methodologies, philosophical approaches, and best practices that still wouldn't yield a single "perfect" way to create one. Rather than getting entangled in theoretical perfection or philosophical accuracy, here's a practical step-by-step approach to create a working ontology quickly and effectively.

Step-by-Step Ontology Design Process

Step 1: Define Competency Questions

Begin by establishing what your knowledge graph needs to answer. The knowledge engineering community has converged on competency questions as a schema determination standard. Write down the factual questions about entities that your system must be able to answer. These might include questions like "What ordinances were passed in 2024?", "Which agenda items resulted in approved resolutions?", or "What documents reference a specific property address?" These questions guide your ontology design by revealing the required entities, relationships, and attributes your system needs to capture.

Step 2: Identify Core Components

You need to systematically define four core components of your knowledge graph. First, identify the classes or entity types that will form the backbone of your graph. For each class, determine what characteristics and criteria an entity must satisfy to belong to that class, and establish identity criteria that make any two entities distinct. For example, "Ordinance" might be a class where the identity criterion is the unique ordinance number.

Next, define the relations that will connect these entities. For each relation, you'll need to specify the domain and range classes, essentially determining what types of entities the relation connects and in which direction. You'll also establish instantiation criteria, defining what characteristics entities need to satisfy to be related through this relation. Be aware that database choice matters here: Neo4j supports attributes on

relations themselves, standard RDF does not, though RDF* does. This capability can be crucial for capturing temporal or contextual information about relationships.

Then specify the attributes or properties your entities will have. These properties capture the detailed information about each entity that doesn't warrant creating separate entities and relationships. Finally, define the constraints and rules that will govern these entities and relationships to ensure data consistency and domain validity.

Step 3: Draft Initial Framework

Create your first conceptual framework by writing down the entities, relationships, and attributes you know are important to your application. If you're unsure about these elements, consult with subject matter experts who can provide domain-specific insights. Focus on capturing the obvious, essential components first rather than trying to be exhaustive in this initial pass.

Step 4: Leverage LLM Assistance

Once you have your initial framework, you can refine and expand your ontology using large language models. Feed your initial ideas along with application-related documents to an LLM, providing it with the necessary context to understand your domain. Ask the LLM to help you create a structured ontology that includes different entity types, relationships between them with proper domain and range specifications, attributes, and relationship directions. The LLM can often identify patterns and relationships you might have overlooked in your initial draft.

Step 5: Capture Domain-Specific Context

In certain cases, you'll need to add context through custom taxonomies that capture domain-specific structures. When working with municipal documents, for instance, you might need to define a hierarchy of documents, sections, and agenda items, along with how these relate to commission meetings and external items. This structural information is crucial because it represents context that LLMs might not automatically extract. Often, this is knowledge that's self-evident to people who work in the domain but isn't explicitly documented anywhere. Bringing this implicit context to the surface and integrating it into your application enables the system to reason about relationships that humans naturally expect to query across, even when these relationships aren't explicitly stated in any document.

Step 6: Merge and Refine

The next phase involves combining your LLM-assisted ontology with any semi-manual ontologies or taxonomies you've created. Once merged, iterate with subject matter experts to refine the combined ontology until it represents a reasonably accurate model of your domain. This iterative refinement is where domain expertise becomes invaluable, as experts can identify missing relationships, incorrect categorizations, or oversimplifications that need adjustment.

Step 7: Validate Against Competency Questions

Test your ontology by verifying it can answer all the competency questions you defined in Step 1. Check that your ontology contains the main types of entities in your domain, the primary relationships between entities including their directionality and cardinality, and the key attributes for each entity type. If you find gaps where certain questions cannot be answered, adjust your ontology accordingly. This validation step often reveals missing elements that seemed unimportant initially but are actually crucial for your use cases.

Important Considerations

Remember that creating a useful ontology is inherently an iterative process that will continue to evolve as your organization changes. It's important to recognize the philosophical limitation here: you cannot capture the entire complexity and nuances of your application domain on paper, no matter how thorough you try to be. Rather than stressing about creating a perfect ontology, prioritize building something useful that can be effectively employed in daily operations.

The resulting ontology should enable both humans and machines to understand and reason about your domain information while remaining flexible enough to accommodate future growth and change. This balance between completeness and practicality is what transforms an academic exercise into a valuable operational tool.

Scaling Up: The Graph-of-Graphs Architecture

Modeling the entirety of the city's data as one huge, functional, and coherent knowledge graph at once is too big of a task to be undertaken. However, a more feasible approach is to model each sub-system (or data source) as its own knowledge graph, effectively creating a higher-level system that is a "graph of graphs." This concept is analogous to the structure of relational databases.

In this model, an overarching "ontology of the truth" would describe each individual graph as an entity with its own attributes. This overhead system would know what information can be extracted from each sub-graph, allowing an LLM to form precise queries about where to pull specific information. To facilitate this, each piece of data is effectively stored not as a triple but as a quadruple: subject, predicate, object, and graph of belonging.

Although it would not be just one huge graph, this structure would require a significant level of standardization for the system to work cohesively.

Implementation Pipeline

Building such a system would follow a structured process:

Identify and Compartmentalize: The first step is to list all available data sources and then compartmentalize them into specific domains to create a list of all the sub-graphs needed. Each city system or domain (e.g., transportation, public works, emergency services) will have its own graph.

Build Domain-Specific Ontologies: For each domain, relevant sources of information (like organizational charts, specific databases, legal documents, etc.) must be gathered. From these sources, an ontology is

created, defining the key entities, their attributes, and relationships. This process can be done manually with the assistance of an LLM to suggest an initial structure.

Create the Master Ontology & Metadata Hub: Once the individual ontologies are mapped out, they are integrated into a master "ontology of ontologies." This functions as one of the metadata hubs that create an organization-wide map. This map holds crucial information about the data's shape, location, the systems and pipelines that process it, and its ultimate consumers. This hub is crucial for orchestrating the sub-graphs and enabling the retrieval of information from multiple sources. By connecting all the individual graphs through this master ontology, the whole becomes greater than the sum of its parts.

While each sub-graph would have its own ontology according to the domain represented, the idea is that some parts of the ontology can be common. For example, a "document" will be a document regardless of whether it is for the fire department or for the city clerk, so there will be common attributes of a document property. To manage these common elements effectively, the system must create a persisted record of master entities. This practice is essential for entity resolution tasks: the process of identifying and linking records that refer to the same real-world entity (e.g., the same address or person) across different data sources.

The ontology of each sub-graph (for example, the fire department's) would then have specific entities that would appear only in that ontology (e.g., properties of the object "fire-hydrant") or the specific things needed for the fire department.

Graph Construction Pipeline Example

- **Seed ontology in:** The user provides an ontology.
- **Extract:** That ontology drives extraction modules (PDF, ArcGIS, image/OCR, etc.) to run ontology-guided NER and relation extraction.
- **Model business logic:** In parallel, define a lightweight taxonomy/custom ontology for domain rules (e.g., document hierarchies).
- **Merge:** Combine the NER-derived ontology with the business taxonomy to form a higher-level ontology.
- **Schema & load:** Generate a schema from this ontology and load it into the graph database.
- **Query agent:** An agent/workflow maps natural-language questions to graph queries, optionally augmented by vector retrieval.
- **Answer:** An LLM converts query results into clear natural-language answers, adding brief reasoning when needed.

Working with Graphs

The end product to wrangle and tame the knowledge graphs should be as little brittle as possible, but it probably looks like a library. The library would contain functions that would allow for knowledge graph incremental updates, nodes deduplication and disambiguation to make the graph more precise as it grows, ontology correction functions, etc.

The library would contain a standard pipeline for knowledge graph construction and querying, as well as methods that are used to add new batches of data, update and modify the knowledge graphs. Providing methods to modify and update the graphs using an abstracted library function would be a major advantage as when the graph ontology is modified, it would be possible to modify it and re-run it through an already existing

pipeline instead of having to dive in the codebase's brittle logic each and every time an ontology update needs to happen.

With these foundational concepts established, we can now present the complete blueprint for implementing a production-ready municipal knowledge system that combines the linguistic capabilities of LLMs with the structural guarantees of knowledge graphs.

Part V: The Unified-Graph Municipal Knowledge System

Application Blueprint:

Unified-Graph Municipal Knowledge System

How to read this

Layer 1 gives an executive snapshot and quick read.

Layer 2 shows the full system at a glance with diagrams and outputs.

Layer 3 walks the end-to-end workflow and the seven operational stages.

Layer 4 is the detailed reference, including ontology, metadata, IDs, LLM specifics, dedup and merge, graph build, performance, and outputs.

Layer 5 covers Querying capabilities end to end, including the 4 query types, routing, fusion, ranking, and confidence.

Layer 1. Executive snapshot and quick read

What this is

A data processing pipeline that transforms unstructured city government documents such as PDFs into:

- A structured and queryable knowledge graph in Cosmos DB Gremlin
- A searchable vector index in Azure Cognitive Search
- JSON manifests for entities, relationships, and a merge map

Why it works

A sequential refinement philosophy that progressively builds precision:

- **Taxonomy Graph (The Skeleton)** extracts high precision from structured elements such as agenda titles and item codes
- **NER Graph (The Flesh)** adds broad coverage from Named Entity Recognition over full text
- Both are unified through deduplication and merge

The 3 main phases, 7 stages, 4 ontology splits and 2 merge priorities

- **3 directories:** phase1_preprocessing, phase2_building, phase3_querying
- **7 stages:** Preprocessing → JSON→Markdown → Taxonomy → NER → Dedup → Graph Push → Vector Push
- **4 LLM ontology splits** per text chunk
- **2 merge priorities:** taxonomy > ner

End-to-end at a glance

City PDF → Knowledge Graph plus Vector Index

Workflow map: Stage 1: Preprocessing → Stage 2: JSON→Markdown → Stage 3: Taxonomy → Stage 4: NER → Stage 5: Dedup → Stage 6: Graph Push → Stage 7: Vector Push

Each section starts with a **You are here** badge so readers can keep their place.

Sequential refinement approach

- Taxonomy extraction from structured JSON for high precision
- NER extraction from markdown for broad coverage
- Sequential processing with Phase 1 entities feeding into NER for context
- Unified through deduplication and merge

Visual workflow map

Pipeline Overview

This data processing pipeline transforms raw documents through multiple stages of extraction, enrichment, and storage:

Stage 1: Preprocessing

Initial document discovery, OCR/parsing, JSON extraction, metadata enrichment, and chunking with quality gates.

Stage 1.5: JSON→Markdown

Converts extracted JSON content into markdown format for further processing.

Stage 3: Taxonomy

Extracts structured information including agenda items, legal policies, and document links.

Pipeline Overview

This data processing pipeline transforms raw documents through multiple stages of extraction, enrichment, and storage:

Stage 1: Preprocessing

Initial document discovery, OCR/parsing, JSON extraction, metadata enrichment, and chunking with quality gates.

Stage 1.5: JSON→Markdown

Converts extracted JSON content into markdown format for further processing.

Stage 3: Taxonomy

Extracts structured information including agenda items, legal policies, and document links.

What comes out

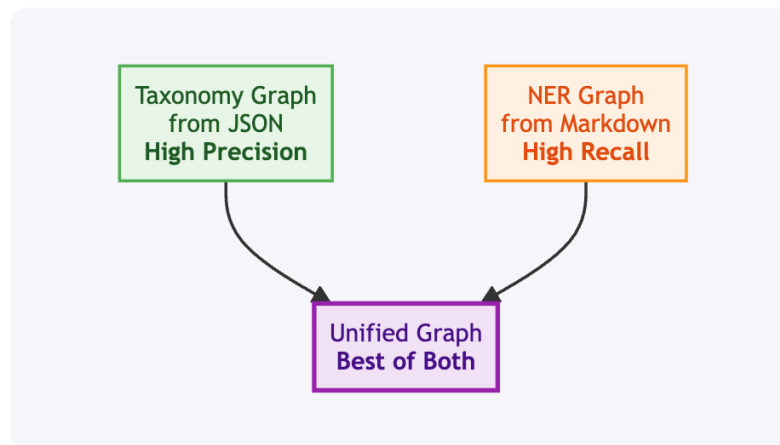
- **Cosmos DB Gremlin graph** for complex queries
 - Example query: Which people voted on the policy that amended this contract
 - **Merged JSON manifests** for entities and relationships
 - **Vector index in Azure Cognitive Search** for semantic search
-

Layer 2. System at a glance

Sequential processing stages

- **Stage 1 (Preprocessing)**: PDFs → structured JSON with 3-stage extraction
- **Stage 2 (Conversion)**: JSON → Markdown for NER processing
- **Stage 3 (Taxonomy)**: Deterministic parsing of structured JSON, authoritative IDs, high precision
- **Stage 4 (NER)**: Azure OpenAI GPT-4.1 nano reads markdown text to capture implicit entities and relationships, high recall
- **Stage 5 (Deduplication)**: Merge taxonomy and NER entities
- **Stage 6 (Graph Construction)**: Push to Cosmos DB
- **Stage 7 (Vector Push)**: Index chunks in Azure Search

Graph Unification Architecture



Graph Unification Strategy:

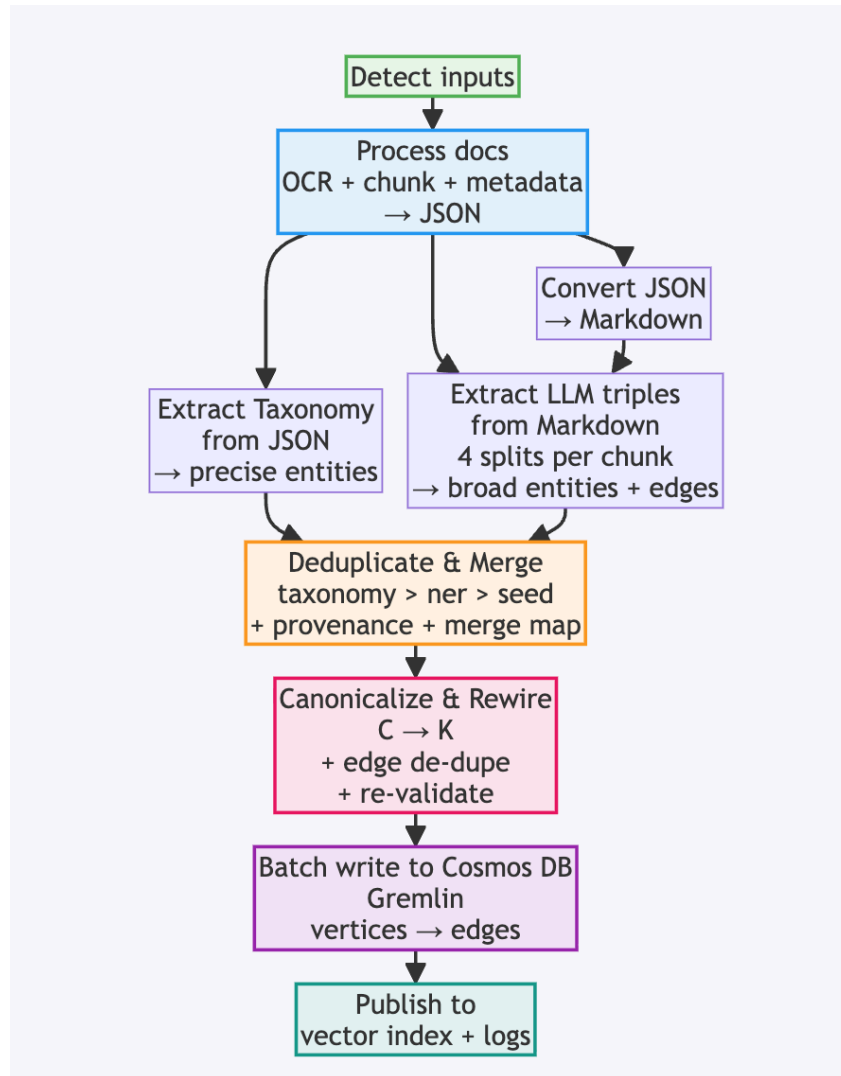
- **Taxonomy Graph:** Extracted from structured JSON with high precision - fewer but more accurate entities and relationships
- **NER Graph:** Extracted from Markdown text with high recall - captures more entities but may include some noise
- **Unified Graph:** Combines both sources to achieve optimal balance between precision and recall

LLM integration

- **Unified triple extraction** returns Subject, Predicate, Object plus attributes and provenance via four parallel calls per chunk, one per ontology group
- **Parallel ontology splitting:** each chunk uses **4 focused prompts**
 - **Group 1: People and Roles** (Person, Role, Organization)
 - **Group 2: Governance and Actions** (Meeting [stored as Event], Event, Action, VoteOutcome, Board)
 - **Group 3: Documents and Policies** (Document, AgendaDocument, Policy, Contract, LegalReference, Section)
 - **Group 4: Content and Resources** (AgendaItem, Presentation, PublicComment, Project, Location, Asset, Technology, Topic, Appointment)

- Divide and conquer acts like AI blinders that improve accuracy

One-screen schema



Layer 3. End-to-end workflow and stages

Pipeline stages

Seven stages:

1. **Stage 1:** Data Preprocessing (contains complete 3-stage internal pipeline)

2. **Stage 2:** JSON to Markdown Conversion
3. **Stage 3:** Taxonomy Synthesis
4. **Stage 4:** NER Extraction
5. **Stage 5:** Deduplication and Merge
6. **Stage 6:** Graph Construction
7. **Stage 7:** Vector DB Push

Important Note: The code uses internal stage numbers (1.5, 3.5) but we've renumbered them sequentially here for clarity. The actual execution order remains the same.

Three directory modules (legacy naming):

- **phase1_preprocessing/** - Contains Stage 1 and Stage 2 code
- **phase2_building/** - Contains Stages 3, 4, 5, 6, and 7 code (including NER extraction modules)
- **phase3_querying/** - Contains query capabilities (not a pipeline stage)

Stage 1 of 7. Data Preprocessing & Extraction (phase1_preprocessing)

You are here: Stage 1 of 7

Workflow: **Stage 1 (Preprocessing)** → Stage 2 (JSON→Markdown) → Stage 3 (Taxonomy) → Stage 4 (NER) → Stage 5 (Dedup) → Stage 6 (Graph) → Stage 7 (Vector)

What happens

Progressively refine raw PDFs into structured JSON using a 3-stage extraction process.

3-Stage Extraction Process (within Stage 1)

Stage 1 runs the complete preprocessing pipeline which internally contains:

- **Internal Stage 1:** PDF OCR with Docling + PyMuPDF (structural parsing to extract hyperlinks)
- **Internal Stage 2:** LLM agenda extraction with regex fallbacks
- **Internal Stage 3:** Ontology enhancement with entity extraction

Discovery and prioritization

- Scan for new or modified PDFs with checksums and timestamps
- Prioritize partially processed files
- Skip logic commonly avoids reprocessing all already processed files

Digitization and caching

- OCR converts documents to machine-readable text and layout
- OCR output is cached

Extraction to JSON

- Extract text and layout, including OCR when needed
- Build structured JSON with agenda items, sections, and metadata

Deterministic filename linking

- Files like `MM_DD_YYYY - Verbatim Transcripts - <agenda-codes>.pdf` link transcripts to agenda items without LLM
- This is completely deterministic

Stage 1 outputs - the data contract

Document-level metadata from Internal Stage 1 (OCR):

- **doc_id**: Unique document identifier (e.g., "DOC_abc123def456")
- **Source_File_Name**: Original filename
- **source_file_path**: Full path to source file
- **full_text**: Extracted text content
- **pages**: [] (array, pages extraction)
- **hyperlinks**: Array with {url, text, page, coordinates}
- **metadata**:
 - **extraction_method**: "docling_ocr_pymupdf" or "skipped_already_processed"

- **page_count**: Number of pages from PDF
- **total_chars**: Character count of **full_text**
- **hyperlink_count**: Number of hyperlinks found
- **extraction_timestamp**: ISO timestamp
- **provenance**: Records extraction tools (Docling, PyMuPDF) and whether fallbacks were taken

Additional metadata added in subsequent internal stages:

- **Document_Type**: Normalized to enum (Agenda | Verbatim | Ordinance | Other) - added during classification
- **processing_status**: Tracks pipeline progress
- **checksum**: For change detection
- **discovery_time** and **processed_time**: Timestamps for tracking

Internal Stage 2 within Stage 1: Agenda Structure Extraction

This happens inside the Stage 1 pipeline for agenda files:

Extracted metadata:

- **meeting_date**: Extracted from document content
- **sections**: Array of {**section_name**, **section_id**, **start_line**, **end_line**}
- **agenda_items**: Array with:
 - **item_code**: (e.g., "E-1", "F-10")
 - **document_reference**: (e.g., "23-6784")
 - **title**: Item title text
 - **section_name**: Parent section

Locale-aware date normalization:

Ambiguous inputs such as 01.09.2024 standardized to ISO YYYY-MM-DD using document locale rules while keeping the raw string

This data is passed to Internal Stage 3 (within Stage 1) for ontology enhancement.

Internal Stage 3 within Stage 1: Ontology Enhancement

This stage enhances the agenda structure and saves enhanced JSON files:

Output: Enhanced JSON files saved to organized subdirectories:

- **agenda/** - Enhanced agenda files
- **legal/** - Enhanced ordinances and resolutions
- **verbatim/** - Processed transcript files

Upstream quality gates and validation

- **Gate A:** Structural fidelity. All top-level headings present. Tables round-trip to Markdown grids. Anchors are unique
- **Gate B:** Metadata completeness. Required fields present such as doc_id, source_file_name, source_file_path, full_text
- **Gate C:** JSON structure integrity. Valid JSON with expected schema
- **Gate D:** Ontology enhancement. Enhanced JSON files prepared for downstream processing
- **Gate E:** Provenance and traceability. doc_id consistent. Extraction metadata complete

Why Stage 1 matters

- Structured JSON enables precise taxonomy extraction
- Metadata normalization ensures consistency
- Traceability from provenance and source tracking
- Sets foundation for all downstream processing

Stage 2 of 7. JSON to Markdown Conversion (phase1_preprocessing)

You are here: Stage 2 of 7

Workflow: Stage 1 (Preprocessing) → **Stage 2 (JSON→Markdown)** → Stage 3 (Taxonomy) → Stage 4 (NER) → Stage 5 (Dedup) → Stage 6 (Graph) → Stage 7 (Vector)

What happens

Convert structured JSON from Stage 1 into Markdown format for NER processing

Inputs

JSON files from Stage 1 in organized subdirectories:

- `stage1/`, `stage2/`, `stage3/` - Intermediate processing stages
- `verbatim/` - Verbatim transcripts
- `legal/` - Ordinances and resolutions

Processing

- Extract text content from JSON
- Format as clean Markdown preserving headings, lists, and structure
- Maintain metadata linkage to source JSON

Outputs

- Markdown files in `extracted_markdown/` directory
- One-to-one mapping with source JSON files

Why Stage 2 matters

- NER extraction requires text format, not JSON
- Markdown preserves document structure for context
- Enables chunk-based processing in Stage 4

Stage 3 of 7. Taxonomy Synthesis (phase2_building)

You are here: Stage 3 of 7

Workflow: Stage 1 (Preprocessing) → Stage 2 (JSON→Markdown) → **Stage 3 (Taxonomy)** → Stage 4 (NER) → Stage 5 (Dedup) → Stage 6 (Graph) → Stage 7 (Vector)

What happens

Parse structured JSON from Stage 1 to create typed entities with deterministic IDs

Store in `simple_ner_graph/registry/` directory (same structure as NER will use)

Note: This is different from the ontology enhancement in Stage 1 (Internal Stage 3). Stage 3 creates the authoritative taxonomy entities that will be used as the skeleton of the graph.

How Stage 3 differs from Stage 1's internal step

- **Stage 1, internal step 3:** Enhances the agenda structure and saves organized JSON files.
- **Stage 3:** Creates the authoritative entities with deterministic IDs, standardized metadata, and complete provenance tracking.

Stage 3 entity example:

```
{  
  "personID": "person_john_smith",  
  "firstName": "John",  
  "lastName": "Smith",  
  "title": "Commissioner",  
  "type": "Person",  
  "Meeting_Date": "2024-01-09",  
  "source_file_name": "agenda_01_09_2024.json",  
  "_sources": "taxonomy_agenda"  
}
```

Inputs

- **Agenda JSON** produces **Meeting, Section, Agendaltem** entities

- **Legal documents** produce **Policy** entities such as **Ordinance** or **Resolution** with official numbers and titles
- **Verbatim transcripts** produce **Document** nodes linked to the right agenda items via deterministic filename pattern

LLM integration with guardrails and fallbacks

- Extract sections, item codes such as **E-1**, document references such as **23-6819**, complete legal titles without truncation, sponsors, motions, and vote summaries
- Regex and rule-based fallbacks provide minimum viable structure if LLM calls fail

Deterministic, date-aware IDs

- `agenda_item_<item_code>_<MM_DD_YYYY>` such as `agenda_item_e1_01_09_2024`.
Normalized date form also supported internally for joins. This means the system supports multiple date formats for the same agenda item to enable proper matching and deduplication
- Internal joins support `agenda_item_<item_code>_<MM_DD_YYYY>`
 - Example: `agenda_item_e4_2024_01_09` and `agenda_item_e4_2024_02_13` are different meetings with the same code
- `policy_<type>_<year>_<number>` such as `policy_ordinance_2024_01`
- `policy_ordinance_2024_01_[hash8]` to avoid collisions
- `person_{firstName}_{lastName}` such as `person_john_smith`. NEVER include titles in person IDs to avoid ID instability when roles change. Use collision suffixes for duplicate names:
`person_john_smith_2`

Metadata added by Stage 3

For each entity created:

- **Meeting_Date**: From agenda extraction (ISO format)
- **Document_Type**: Classification (Agenda, Ordinance, Resolution, etc.)
- **committeeOrBody**: Extracted from agenda header

- **policyNumber:** For Policy entities
- **source_file_name/path:** Inherited from Stage 1
- **_sources:** "taxonomy_agenda" or similar to track origin

Output location

- Entities written to `simple_ner_graph/registry/{EntityType}/` directories
 - Relationships written to `simple_ner_graph/registry/relationships/`
-

Stage 4 of 7. NER Extraction (phase2_building)

You are here: Stage 4 of 7

Workflow: Stage 1 (Preprocessing) → Stage 2 (JSON→Markdown) → Stage 3 (Taxonomy) → **Stage 4 (NER)** → Stage 5 (Dedup) → Stage 6 (Graph) → Stage 7 (Vector)

What happens

- Process Markdown from Stage 2 using Phase 1 entities as context
- Each Markdown chunk is processed by four parallel LLM calls, one per entity group:
 - **Group 1: People and Roles** (Person, Role, Organization)
 - **Group 2: Governance and Actions** (Meeting [stored as Event], Event, Action, VoteOutcome, Board)
 - **Group 3: Documents and Policies** (Document, AgendaDocument, Policy, Contract, LegalReference, Section)
 - **Group 4: Content and Resources** (AgendaItem, Presentation, PublicComment, Project, Location, Asset, Technology, Topic, Appointment)
- Single-pass triple extraction yields typed entities with standardized IDs and relationships with attributes and full provenance

Chunking strategy

- Markdown is chunked into about **1,500 characters** with **about 10 percent overlap**

- Character limit (or token limit) to prevent JSON parsing failures from oversized LLM responses
- Sentence-aware chunking that doesn't split tables, list items, or legal titles
- Chunks saved to `simple_ner_graph/document_chunks/` directory as .txt files

Chunking and processing approach

- Uses Phase 1 entities for logging purposes (about 50-100 entities typically logged)
- Processes markdown chunks independently using 4 parallel ontology groups
- Chunk filenames: Named using the format `{chunk_id}.txt` where `chunk_id` is a unique identifier (often a hash-based ID like `chunk_abc123def456.txt`), with each file containing a metadata header followed by the chunk content separated by a `---` delimiter. The chunk files have this structure:

```
# Chunk ID: chunk_abc123def456
# Document: agenda_01_09_2024.json
# Document Type: agenda
# Meeting Date: 01.09.2024
# Index: ⅓
# Source: city_clerk_documents/extracted_json/agenda/agenda_01_09_2024.json
-
[Actual chunk text content here]
```

LLM guardrails and fallbacks

- **Ontology-aligned emission** ensures required ID fields and attributes are present and aligned
- **Reading-status classification** (such as ordinances first versus second reading) uses LLM classification with deterministic fallbacks based on section headers and formulaic phrases
- **Rule-based fallback** keeps a baseline when an LLM call fails
- **API rate limiting** is handled at runtime with configurable delays between calls (not stored as metadata)
- **Pre-flight token counting** prevents exceeding context limits
- **Regex and rule-based fallbacks** provide minimum viable structure if LLM calls fail

Metadata added by Stage 4

Each extracted entity includes:

- **extraction_chunk_id**: Which chunk it came from (for debugging).
- **_sources**: "ner_chunk_X" to track NER origin. The _sources field is particularly important because it determines merge priority during deduplication - entities with taxonomy_ sources get priority over ner_ sources when merging duplicates.
- Confidence scores (internally used, not stored)

Example Stage 4 (NER) entity:

```
{  
  
  "personID": "person_john_smith",  
  
  "firstName": "John",  
  
  "lastName": "Smith",  
  
  "title": "Commissioner",  
  
  "type": "Person",  
  
  "votePosition": "yes",  
  
  "extraction_chunk_id": "chunk_transcript_01_09_2024_005",  
  
  "_sources": "ner_chunk_5",  
  
  "source_file_name": "01_09_2024_verbatim_transcript.txt"  
  
}
```

What Stage 4 adds vs Stage 3:

- **Stage 3**: Basic taxonomy entity with ID and metadata
- **Stage 4**: Four parallel ontology-focused LLM calls per chunk, each specialized for domain expertise
 - Per-chunk entity extraction with no awareness of other chunks or documents
 - Attribute enrichment beyond basic taxonomy using contextual analysis

- Implicit relationship discovery within chunk boundaries (organizational membership inferred from titles, procedural relationships from discussion context)

Output location

- Entities written to `simple_ner_graph/entities/{EntityType}/` directories
- Relationships written to `simple_ner_graph/relationships/`
- Indices built in `simple_ner_graph/indices/`

Why this stage exists

- Captures entities and relationships found only in body text or transcripts outside agenda header structure
- Uses Phase 1 entities as context for consistency

Stage 5 of 7. Multi-Source Deduplication (phase2_building)

You are here: Stage 5 of 7

Workflow: Stage 1 (Preprocessing) → Stage 2 (JSON→Markdown) → Stage 3 (Taxonomy) → Stage 4 (NER) → **Stage 5 (Dedup)** → Stage 6 (Graph) → Stage 7 (Vector)

Goal

Unify Taxonomy's authoritative entities (from Stage 3) with NER's broad entities (from Stage 4) into a single canonical set

Triplet-based deduplication complexity

- When an entity is created multiple times, we want to keep only one version of the entity to avoid duplicate data in the system and to make it so that each entity in our application truly possesses all the relationships associated with that entity across all documents.
- Extracting triplets is crucial for creating knowledge graphs, but deduplicating entities within the application is an equally vital step. We use "canonical" entity to describe the entity that is retained in the application (as maybe it already is in the graph database) and "candidate" entity to refer to an entity that

was extracted but is not part of the graph yet.

- Because we store triplets, each triplet is made up of 2 "candidate" entities and one relationship that connects them. Therefore, there are 3 deduplication scenarios:
 - 1) All candidate entities have no duplicate and the entire triplet can be pushed to the database
 - 2) One of the candidate entities has a duplicate (therefore we need deduplication over that entity) but the second entity has no duplicate in the graph database
 - 3) Both of the candidate entities have duplicates and therefore we need deduplication over both of them
- This triplet-based complexity drives the sophisticated relationship rewiring and endpoint management described below.

EntityDeduplicatorExtended

- **Name normalization:** Strip titles, standardize casing. Harmonize attribute names such as meetingDate versus meeting_date
- **Similarity scoring** that is configurable:
 - **60 percent** name similarity
 - **30 percent** attribute overlap
 - **10 percent** contextual similarity
- **Two-stage thresholds:**
 - Candidate generation around 0.85 to collect plausible matches (configurable via ENTITY_CANDIDATE_THRESHOLD env var)
 - Final merge at 0.95 or higher to avoid over-merging legal records (configurable via ENTITY_FINAL_THRESHOLD env var)
- **Merge priority:** taxonomy over ner. Taxonomy remains authoritative. NER fills missing attributes and links
- **Missing endpoints:** Auto-create minimal stubs for referenced but unseen nodes that can be enriched later

- **Incremental support:** Version tracking, conservative merges that preserve existing values, list-property union, and conflict logs
- **Relationship rewiring:** Updates relationship endpoints to point to canonical entity IDs, tracking rewired edges, unresolved references, and duplicate relationship removal
- **Opportunity for attribute enrichment through intelligent merging:** Rather than simply discarding duplicate entities, the system enriches canonical entities (from both taxonomy and NER sources) by selectively incorporating valuable attributes from duplicate candidates when confidence thresholds are met, creating more complete entity profiles while maintaining source-based priority rules

Outputs

- Merged Entity Manifests at `simple_ner_graph/merged/entities/{Type}.json`
- Relationship Manifest at `simple_ner_graph/merged/relationships.json`
- Merge Map at `simple_ner_graph/merged/merge_map.json` mapping `old_id` → `canonical_id`

Stage 6 of 7. Graph Construction (phase2_building)

You are here: Stage 6 of 7

Workflow: Stage 1 (Preprocessing) → Stage 2 (JSON→Markdown) → Stage 3 (Taxonomy) → Stage 4 (NER) → Stage 5 (Dedup) → **Stage 6 (Graph)** → Stage 7 (Vector)

Validation before write

- **Schema validation:** Required fields present and correctly typed
- **Relationship validation:** Enforce ontology constraints such as `Section hasAgendaItem` → `AgendaItem`. Reject invalid patterns such as `Person` → `Section`
- **Sanity checks:** No orphans. Unique IDs. Referential integrity

Graph build with CustomGraphBuilder to Cosmos DB Gremlin

- Batch writes about **500 items per batch**, parallel execution, connection pooling, retry with exponential backoff, automatic reconnection on transient failures
- On validation failure: log and continue so processing does not halt
- **Provenance everywhere:** Each node and edge carries `source_file_name|path`, `_sources` such as `taxonomy_agenda` or `ner_chunk_5`, `extraction_chunk_id`, and tool or version

Packaging and downstream

- Structured entities and relationships to **Cosmos DB Gremlin**
 - All validation logs and statistics tracked for observability
-

Stage 7 of 7. Vector DB Push (phase2_building)

You are here: Stage 7 of 7

Workflow: Stage 1 (Preprocessing) → Stage 2 (JSON→Markdown) → Stage 3 (Taxonomy) → Stage 4 (NER) → Stage 5 (Dedup) → Stage 6 (Graph) → **Stage 7 (Vector)**

What happens

Push document chunks with metadata to Azure Cognitive Search for semantic search

Inputs

- Document chunks from [simple_ner_graph/document_chunks/](#)
- Merged entity mappings for canonical IDs (mapping table that tracks how duplicate entity IDs get resolved to single canonical (authoritative) IDs during the deduplication process in Stage 5)

Processing

- Generate embeddings for chunk text
- Include metadata for filtering and faceting
- Map entity references to canonical IDs

Outputs

- Searchable vector index in Azure Cognitive Search
 - Enables semantic search over document content
-

Layer 4. Detailed reference

Foundations first: data model, ontology, taxonomy, metadata, identifiers

Ontology and taxonomy

- **Ontology** is the system's blueprint. Defines core nouns, verbs, adjectives
 - **Entities (Nouns):** Person, Organization, Document, Policy, Event, Action, Asset, Project, Location, Role, Topic, Agendaltem, Section, Meeting, AgendaDocument, Contract, LegalReference, VoteOutcome, Board, Technology, Presentation, PublicComment, Appointment
 - **Attributes (Properties):** Descriptive facts such as item_code for Agendaltem or firstName/lastName for Person (name computed as alias)
 - **Relationships (Verbs):** Connections such as a Person can SPONSOR an Agendaltem
- **Taxonomy** is a hierarchical structure derived directly from documents. Example: a Meeting contains Sections that contain Agendaltems. Provides the authoritative backbone for the Taxonomy Graph

Data model and ontology, concise but actionable

- **Purpose:** Provide a typed model that the graph can enforce and extraction can populate consistently
- **Core entity classes (actual scope from UnifiedOntology):**
 - **People & Organizations:** Person (canonical: firstName, lastName; name as computed field), Organization, Role
 - **Documents & Legal:** Document, AgendaDocument, Policy, Contract, LegalReference
 - **Meetings & Structure:** Meeting (stored as Event), Section, Agendaltem, Board
 - **Actions & Outcomes:** Event, Action, VoteOutcome
 - **Resources & Context:** Asset, Project, Location, Technology, Topic
 - **Participation:** Presentation, PublicComment, Appointment
- **Person attributes:** firstName and lastName are canonical fields for query matching; name field computed as alias for display
- **Attributes:** Typed key-value facts on entities and edges such as ordinance numbers, agenda codes, titles, dates, outcomes
 - Attributes contribute 30 percent to dedup scoring

- **Relationships (edges):** Typed predicates constrained by the ontology
 - Examples that are valid:
 - Section hasAgendaItem → AgendaItem
 - AgendaItem implements → Policy
 - Edges carry attributes such as vote details and provenance
- **The rules (ontology)** define classes, predicates, and allowed source → predicate → target patterns
 - Used for extraction focus, validation, and graph build
- **Taxonomy** maps structured JSON to ontology classes, provides deterministic IDs and high-precision attributes. When merging, Taxonomy wins over NER

Metadata and provenance

Traceability is critical. Every piece of data is stamped with metadata to track origin, processing stage, and timestamps

Standard metadata on every artifact, normalized end-to-end:

- **Core identification:**
 - `source_file_name|path`
 - `doc_id` (unique document identifier)
 - `extraction_method` and `extraction_timestamp`
 - `processing_status`
- **Document classification** (added progressively):
 - `Document_Type` normalized to an enum such as Agenda, Verbatim, Ordinance, Other
 - `Meeting_Date` normalized to ISO with `meetingDate_raw` kept
- **Provenance tracking:**
 - Module or stage and tool such as Docling, PyMuPDF, LLM, regex

- Whether fallbacks were taken
- Versioning and timestamps such as discovery time and processed time
- Checksum for change detection
- **Context and relationships:**
 - Reading status such as ordinances first versus second derived via LLM with rule-based fallback
 - Linkage context including parent IDs such as Meeting and Section
 - Related agenda codes and referenced document numbers
 - committeeOrBody, meetingLocation (from agenda extraction)
- **Processing tracking:**
 - _sources: Tracks origin (e.g., "taxonomy_agenda", "ner_chunk_5")
 - extraction_chunk_id: For NER entities
 - Processing status and timestamps
- **Content metadata:**
 - Reading status such as ordinances first versus second (via LLM with fallbacks)
 - Linkage context including parent IDs, agenda codes, document numbers
 - Version tracking for incremental updates

Metadata enrichment across stages

- **Stage 1 (Internal Stage 1):** Basic extraction metadata (source file, doc_id, extraction method)
- **Stage 1 (Internal Stage 2):** Agenda-specific metadata (meeting_date, sections, agenda_items)
- **Stage 1 (Internal Stage 3):** Ontology enhancement and JSON file organization
- **Stage 3:** Entity type classification, deterministic IDs, taxonomy relationships
- **Stage 4:** NER enrichment with entity attributes, implicit relationships

- **Stage 5:** Merge metadata (merge_map, canonical IDs, similarity scores)

Deterministic and date-aware IDs

Unique, predictable IDs based on salient features:

- `agenda_item_<item_code>_<YYYY_MM_DD>` → `agenda_item_e1_2024_01_09`
- Internal joins support `agenda_item_<item_code>_<MM_DD_YYYY>`
 - Example: `agenda_item_e4_2024_01_09` and `agenda_item_e4_2024_02_13` are different meetings with the same code
- `policy_<type>_<year>_<number>` → `policy_ordinance_2024_01`
- `policy_ordinance_2024_01_[hash8]` to avoid collisions
- `person_{firstName}_{lastName}` such as `person_john_smith`. NEVER include titles to avoid ID instability when roles change. Use collision suffixes for duplicate names: `person_john_smith_2`

Policy number validation

Cross-validate extracted numbers vs filenames

Example: a file named `2024-02 - 01_09_2024.pdf` vs an extracted 2024-01 leads to correction to 2024-02

Step by step: from raw files to a unified graph

The workflow moves through seven stages. See Stages 1 to 7 above.

Reliability, resilience, performance, and data integrity

Incremental Processing and Recovery

- **Document state tracking:** Processing Registry uses SHA-256 hashes to detect new or modified commission meeting documents automatically, maintaining processing history and status for each file
- **Command line incremental mode:** `--incremental` flag processes only new/modified documents instead of full reprocessing, with specific folder targeting for new meeting directories
- **State persistence across failures:** JSON-based registry with file locking survives failures and enables restart from where previous data push stopped

- **Smart entity merging:** IncrementalEntityMerger preserves existing entities and relationships while adding new ones, using union strategy for list properties and intelligent conflict resolution
- **Version tracking and rollback:** Entity versioning with `_version`, `_created_at`, `_updated_at` metadata enables rollback capability when needed
- **Registry recovery:** Validation detects corruption and falls back to full processing, with rebuild capability to reconstruct processing history from existing documents

Reliability and resilience

- **Fallback mechanisms:** Primary OCR tool **Docling** can fall back to **PyMuPDF**. LLM structure extraction has deterministic regex fallbacks
- **Connection health monitoring:** Transient network issues handled with exponential backoff
- **Automatic retries** with exponential backoff for transient failures
- **Automatic reconnection** on transient failures

Efficiency and performance

- **Incremental processing with skip logic:** Skip documents already `FULLY_PROCESSED`, reducing runtime and cost on subsequent runs
- **Parallel processing:** Concurrency uses tools such as `asyncio.Semaphore` to process multiple documents while respecting quotas
- **Pre-flight token counting** prevents exceeding context limits
- **Batch operations:** Group database operations to reduce API calls and network overhead
 - Graph writes run in about **500-item batches** with connection pooling, backoff, and reconnection

Data integrity

- Hierarchical context preserved: `Meeting` → `Agenda` → `Section` → `Item`
- Validation at each stage against JSON schema
- Referential integrity checks before graph writes

Unified Knowledge Graph equals Taxonomy plus NER

- The Taxonomy extraction from Stage 3 provides the authoritative skeleton backbone
- The NER extraction from Stage 4 fills gaps and implicit links
- Stage 5 Deduplication consolidates both with conservative, provenance-rich merges so legal records are precise and coverage is broad

Outputs and storage

- **Graph:** Cosmos DB Gremlin with validated nodes and edges
- **Search:** Azure Cognitive Search index with chunk text and metadata
- **Artifacts:**
 - `/simple_ner_graph/merged/entities/{Type}.json`
 - `/simple_ner_graph/merged/relationships.json`
 - `/simple_ner_graph/merged/merge_map.json` with `old_id` → `canonical_id`, scores, and reasons
- **Observability:**
 - Validation, merge, and conflict logs
 - Rewiring (updating relationship endpoints (source and target IDs) to point to the canonical entity IDs after deduplication occurs in Stage 5) stats
 - Throughput and accuracy metrics
 - Incremental version records

Appendix. Recap: Stage Framing Aligned to Implementation

Preserves the original descriptions expanded with comprehensive implementation details and actual stage numbering.

Stage 1: Data Preprocessing & Extraction (The Raw Material)

Entry point where raw PDFs are fully processed through a sophisticated 3-stage internal pipeline.

- **Discovery and prioritization:** Scan for new or modified PDFs with checksums and timestamps, prioritize partially processed files, skip logic avoids reprocessing of routine runs
- **Runs complete preprocessing pipeline** which internally contains:
 - **Internal Stage 1:** PDF OCR extraction with **Docling + PyMuPDF** for structural parsing and hyperlink extraction
 - **Internal Stage 2: LLM agenda extraction** with comprehensive regex fallbacks, extracting sections, item codes, complete legal titles, sponsors, motions, and vote summaries
 - **Internal Stage 3: Ontology enhancement** with LLM entity extraction from agenda items, creating Person, Organization, Document, Agendaltem, Meeting, and Policy entities (currently unused by downstream NER)
- **Quality gates:** Structural fidelity, metadata completeness, JSON integrity, ontology enhancement, provenance traceability
- **Outputs:** Structured JSON to organized subdirectories (**agenda/**, **legal/**, **verbatim/**) with comprehensive metadata including doc_id, source file tracking, meeting dates, extraction timestamps, and provenance

Stage 2: JSON to Markdown Conversion (Format Bridge)

Converts structured JSON to LLM-processable Markdown while preserving document integrity.

- **Format transformation:** Converts structured JSON from Stage 1 into clean Markdown format
- **Structure preservation:** Maintains document hierarchy, headings, lists, and metadata linkage
- **Processing enablement:** Creates text format required for Stage 4 NER chunk-based processing
- **One-to-one mapping:** Maintains direct correspondence between source JSON and output Markdown files

Stage 3: Taxonomy Synthesis (The Skeleton)

Sophisticated entity synthesis from structured JSON creating the authoritative knowledge graph backbone.

- **Comprehensive entity creation:** Processes agenda, legal, and verbatim JSON files to create Event, Document, Section, Agendaltem, Person, and Policy entities

- **Deterministic ID generation:** Creates collision-resistant, date-aware IDs with formats like `agenda_item_e1_2024_01_09` and `policy_ordinance_2024_01`
- **Hierarchical relationship building:** Establishes complex relationship chains (Event → hasAgenda → Document → hasSection → Section → hasAgendaItem → AgendaItem)
- **Cross-validation:** Policy number validation against filenames, URL extraction from hyperlinks, sponsorship relationship creation from motion details
- **Registry structure:** Writes to `simple_ner_graph/registry/` using the same directory structure that NER will populate

Stage 4: NER Extraction (The Flesh)

Per-chunk text analysis extracting entities and relationships from narrative content.

- **Chunk-based processing:** Analyzes individual 1,500-character chunks independently with **no cross-chunk awareness**
- **Four parallel LLM calls** per chunk using focused ontology views:
 - **People & Roles:** Person, Role, Organization entities
 - **Governance & Actions:** Meeting, Event, Action, VoteOutcome, Board entities
 - **Documents & Policies:** Document, AgendaDocument, Policy, Contract, LegalReference, Section entities
 - **Content & Resources:** AgendaItem, Presentation, PublicComment, Project, Location, Asset, Technology, Topic, Appointment entities
- **Triple-based extraction:** Subject-predicate-object relationships with full attribute extraction and provenance tracking
- **Narrative intelligence:** Captures implicit entities, contextual attributes, and relationships found only in discussion text and transcripts
- **Output scope:** Entities written to `simple_ner_graph/entities/{EntityType}/`, relationships to `simple_ner_graph/relationships/`, indices to `simple_ner_graph/indices/`

Stage 5: Multi-Source Deduplication (Unifying the Data)

Sophisticated entity and relationship consolidation with intelligent attribute enrichment.

- **Triplet deduplication scenarios:** Because triplets contain 2 entities + 1 relationship, three deduplication cases arise: (1) no duplicates - push entire triplet, (2) one entity duplicate - deduplicate one endpoint, (3) both entities duplicate - deduplicate both endpoints. This complexity necessitates sophisticated relationship rewiring.
- **Multi-source similarity scoring:** Configurable weighted algorithm using **60 percent name similarity**, **30 percent attribute overlap**, **10 percent contextual similarity**
- **Two-stage thresholds:** Candidate generation around 0.85 to collect plausible matches (configurable via `ENTITY_CANDIDATE_THRESHOLD`), final merge at 0.95 or higher to avoid over-merging legal records (configurable via `ENTITY_FINAL_THRESHOLD`)
- **Intelligent merge priority: Taxonomy over NER** with selective attribute enrichment from duplicate candidates when confidence thresholds are met
- **Relationship rewiring:** Updates all relationship endpoints to point to canonical entity IDs, tracking rewired edges, unresolved references, and duplicate relationship removal
- **Missing endpoint handling:** Auto-creates minimal stubs for referenced but unseen nodes that can be enriched later. These are tagged accordingly.
- **Incremental processing:** Version tracking, conservative merges preserving existing values, list-property union, comprehensive conflict logging
- **Output manifests:** Merged entities, relationships, and complete merge map with `old_id` → `canonical_id` mappings, similarity scores, and merge reasoning

Stage 6: Graph Construction (Final Assembly)

Validated graph database construction with comprehensive quality assurance.

- **Pre-flight validation:** Schema validation for required fields, relationship validation against ontology constraints, referential integrity checks, orphan detection
- **Batch optimization:** Writes validated entities and relationships to **Cosmos DB Gremlin** in **500-item batches** with parallel execution, connection pooling, exponential backoff retry, and automatic reconnection
- **Provenance preservation:** Every node and edge carries complete metadata including `source_file_name|path`, `_sources` tracking (`taxonomy_agenda`, `ner_chunk_X`), `extraction_chunk_id`, tool versions
- **Error resilience:** Validation failures logged and skipped to prevent pipeline halt, comprehensive statistics tracking for observability

Stage 7: Vector DB Push (Search Enablement)

Semantic search infrastructure with canonical entity alignment.

- **Chunk indexing:** Pushes document chunks from [simple_ner_graph/document_chunks/](#) to **Azure Cognitive Search** with rich metadata
- **Canonical ID mapping:** Uses merged entity mappings to resolve all entity references in chunks to canonical IDs, ensuring consistency between graph and vector systems
- **Embedding generation:** Creates semantic embeddings for chunk text with metadata for filtering and faceting
- **Cross-modal consistency:** Enables seamless queries across both graph database (precise relationships) and vector index (semantic search) using unified entity identifiers

Pipeline Philosophy:

Sequential refinement from raw PDFs → structured JSON → comprehensive taxonomy skeleton → narrative flesh → unified canonical graph → searchable vector index, with each stage building precision and coverage while maintaining complete provenance and traceability.

Phase 3. Querying (phase3_querying)

Classify → Retrieve → Synthesize. An LLM classifies intent, selects a retrieval plan across a graph architecture plus a vector index, then synthesizes a structured answer with citations and a confidence score.

Router

Four mutually exclusive query types that are collectively exhaustive:

- **SPECIFIC_FACT** → graph lookup in Cosmos DB via Gremlin
- **GENERAL_INFO** → vector search with embeddings
- **COMPLEX_HYBRID** → multi hop hybrid with both graphs and vector
- **UNCLEAR** → disambiguation dialogue, then re route

Hybrid rule of thumb

Graph for precise facts and relationships. Vector for contextual meaning and narrative. Fuse the two, reconcile conflicts, rank, cite, and calibrate confidence.

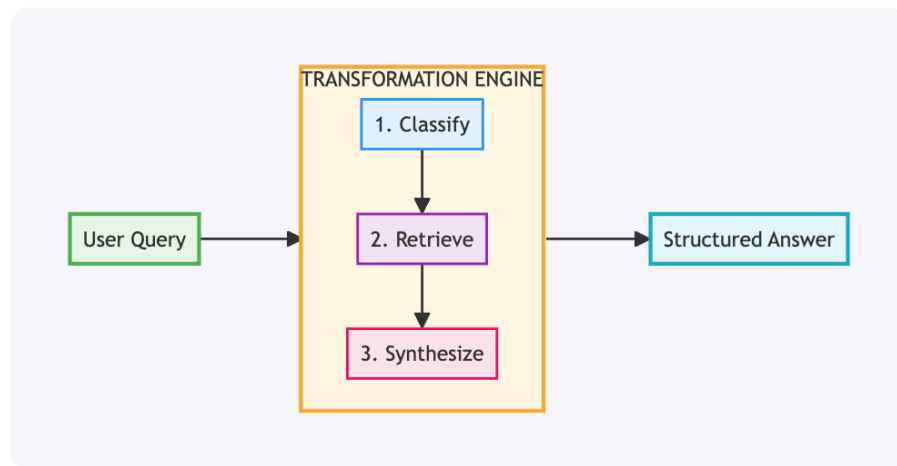
Output format

Executive summary, detailed bullets, inline citations [G#] for graph and [V#] for vector, confidence score.

3.1 Core system and routing diagrams

A. Transformation engine at a glance

Transformation Engine Architecture



Pipeline Stages

1 Classify

Analyzes query intent and categorizes the request type

2 Retrieve

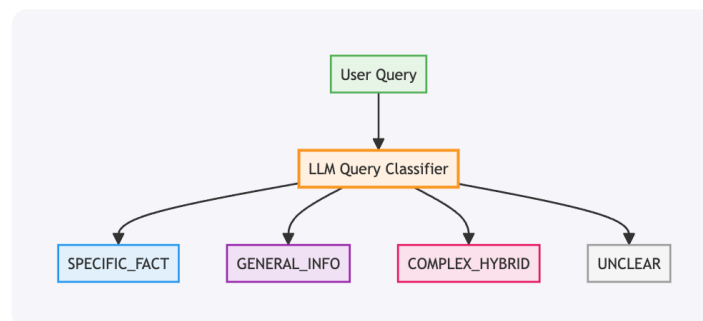
Fetches relevant data from knowledge bases and sources

3 Synthesize

Combines and transforms retrieved data into a coherent response

B. The four path routing system

Query Classification System



Query Categories

SPECIFIC_FACT

Queries seeking discrete, verifiable information

GENERAL_INFO

Broad explanatory or educational queries

COMPLEX_HYBRID

Multi-faceted queries requiring synthesis

UNCLEAR

Ambiguous queries needing clarification

3.2 End to end runtime flow and Inputs, Transformations and Outputs

3.2.1 End to end runtime flow at query time

[1] Receive Query

- Capture raw text plus optional constraints (time range, document type, jurisdiction)

[2] Normalize and Enrich

- Clean text, tokenize, detect identifiers (ordinance numbers, names, dates)
- Date normalization -> ISO plus keep original
- Light semantic hints from taxonomy (synonyms and keywords)

[3] Intent Classify (LLM → 4Q)

- SPECIFIC_FACT | GENERAL_INFO | COMPLEX_HYBRID | UNCLEAR
- Extract candidate entities, relations, temporal cues

[4] Plan and Route

- SPECIFIC_FACT -> Graph plan (Gremlin generated from UnifiedOntology)
- GENERAL_INFO -> Vector plan (embedding plus kNN over chunk metadata)
- COMPLEX_HYBRID -> Hybrid plan (≤ 2 vector hops plus ≤ 2 graph hops, gap driven)
- UNCLEAR -> Disambiguation options -> user selection -> re enter at [3]

[5] Retrieve

- Graph path: execute Gremlin in Cosmos DB under ontology constraints. Collect nodes, edges, attributes, provenance (docId, page, chunkId, headingPath)
- Vector path: embed query (with taxonomy expansion), kNN over chunks and semantic similarity as the underlying similarity measure. Return top k snippets plus metadata (agenda context, speaker labels, policy refs)

[6] Fuse and Reconcile

- Entity de duplication across paths (aliases to canonical IDs)

- Fact reconciliation (Graph facts > Vector narratives for canonical data)
- Where [G#] references a graph-sourced fact from Cosmos DB Gremlin and [V#] references a vector-sourced passage from the semantic index, with each marker resolving to a provenance tuple of (docId, page, chunkId, headingPath).

[7] Rank and Calibrate

- $\text{Ranking} = 0.4 * \text{graph_relevance} + 0.3 * \text{vector_similarity} + 0.2 * \text{temporal_proximity} + 0.1 * \text{entity_authority}$
- Confidence calibration with modifiers for dual path hits, similarity, corroboration, ambiguity

[8] Generate Output

- Executive summary (2 to 3 sentences)
- Structured bullets by topic, entity, meeting
- Inline citations [G#] and [V#] plus confidence score
- If UNCLEAR: present numbered choices and loop back once user selects

[9] Post Processing

- Update caches (embeddings, common graph paths, result sets)
- Store disambiguation mappings (for example “Smith” -> John or Attorney or Director)
- Capture retrieval trace for observability and reproducibility

3.2.2 Inputs, Transformations, Outputs

Inputs (query time)

- User Query: text plus optional constraints (date ranges, document types, jurisdictions)
- Knowledge Stores: Taxonomy Graph in Cosmos DB Gremlin, NER Graph, Vector Index of markdown chunks
- Schemas and Taxonomy: UnifiedOntology for classes, attributes, constraints; Topic taxonomy with hierarchy and synonyms
- Metadata: document level, chunk level, graph level, vector level as produced by the pipeline

- **Config and Policies:** hop limits (≤ 2 graph plus ≤ 2 vector), thresholds, ranking weights, QueryValidator rules
- **Caches:** graph property and path cache, vector top k cache, embeddings cache

Transformations

Normalization → Classification → Routing → Dynamic Gremlin generation → Semantic expansion → Retrieval → Multi hop orchestration → Fusion → Ranking and confidence calibration → Answer synthesis.

- **Query Normalization:** Clean user input, extract entities, standardize date formats
- **Intent Classification:** LLM determines query type (SPECIFIC_FACT, GENERAL_INFO, COMPLEX_HYBRID, UNCLEAR)
- **Routing Decision:** Route to graph-only, vector-only, or hybrid retrieval based on classification
- **Dynamic Query Generation:**
 - **Graph path:** Generate Gremlin queries using LLM with ontology constraints
 - **Vector path:** Create embeddings and execute KNN search with semantic similarity
- **Multi-Hop Orchestration:** For complex queries, execute up to 4 hops with gap analysis between each hop
- **Result Fusion:** Combine graph and vector results, resolve conflicts (graph priority), deduplicate by source
- **Ranking and Scoring:** Apply weighted formula (40% graph relevance + 30% vector similarity + 20% temporal + 10% authority)
- **Confidence Calibration:** Calculate final confidence (base 0.5 + boosts for multiple sources, citations, specificity)
- **Answer Synthesis:** LLM generates natural language response with structured format and inline citations

Outputs

Human readable answer with executive summary and structured bullets. Inline [G#] and [V#] provenance. Confidence score. Optional disambiguation options and machine artifacts for traceability.

3.3 Querying Rules

3.3.1 Query type playbooks

- **SPECIFIC_FACT**

When: single identifiable lookup with clear IDs.

Retrieval path: generate constrained Gremlin from UnifiedOntology, execute in Cosmos DB, collect fact with provenance.

Output: direct fact with [G#].

- **GENERAL_INFO**

When: broad thematic request without specific IDs.

Retrieval path: embed query, vector kNN over chunks with taxonomy expansion, return top k snippets, synthesize.

Output: summary with [V#].

- **COMPLEX_HYBRID**

When: needs both relationships and narrative across entities.

Retrieval path: up to 4 hops total (≤ 2 vector + ≤ 2 graph) driven by gaps. H1 Vector to find candidates → H2 Graph to connect entities → H3 Vector to fill context → H4 Graph to verify identities and counts.

Output: fused answer with [G#] and [V#].

- **UNCLEAR**

When: ambiguous terms or missing constraints.

Retrieval path: detect ambiguity, fetch candidate entities, show numbered options, on selection re enter router.

Output: disambiguation list, then standard answer.

3.3.2 Fusion, ranking, confidence

- Fusion: de duplicate entities, merge complementary facts, resolve conflicts with Graph facts preferred for canonical data, preserve all citations.
- Ranking formula: $\text{score} = 0.4 * \text{graph_relevance} + 0.3 * \text{vector_similarity} + 0.2 * \text{temporal_proximity} + 0.1 * \text{entity_authority}$.
- Confidence: start 0.5, add for dual path corroboration and high similarity, subtract for ambiguity or missing graph corroboration, cap at 1.0.

3.3.3 Simple router heuristics

- If identifiers are present → Graph first.
- If exploratory wording dominates → Vector first.
- If relationship phrasing appears → Hybrid.

- Adaptive rule: start narrow, widen scope only when gaps remain.

3.3.4 Guardrails

- Generate Gremlin only within UnifiedOntology constraints.
 - Normalize dates to ISO and retain originals. No invented dates.
 - Normalize Cosmos results and resolve aliases.
 - Deduplicate by source and canonical IDs.
 - Use fallback layers in order: Graph plus Vector → local NER indices → keyword.
-

Layer 5. Worked mini example

Question

What security measures were discussed in meetings about parking?

Type

COMPLEX_HYBRID

Plan and hops

- H1 Vector: retrieve chunks linking security and parking. Use chunk metadata to surface agenda item context. [V1]
- H2 Graph: traverse Meeting → AgendaItem → Topic. Collect Vote nodes and outcomes. [G1]
- H3 Vector: pull commissioner comments and summaries with speaker labels and headingPath. [V2]
- H4 Graph: verify Person participatesIn Meeting and votesOn Vote identities. Confirm policy sections and counts. [G2]

Fuse, rank, confidence

Merge narrative with canonical graph facts. Prefer graph for canonical fields. Rank by relevance, recency, and authority. Confidence increases when both branches corroborate and citations appear.

PROMPTS:

1) AGENDA EXTRACTION

System Prompt:

""

You are an expert at extracting structured data from city government agenda documents. Return only valid JSON.

""

User Prompt:

""

Extract ALL agenda items from this city council agenda document.

For EACH section/item, extract the following fields:

1. section_name: The name of the agenda section (e.g., "CITY MANAGER ITEMS").
2. item_code: The item code (e.g., "H-1"), normalized to Letter-Number format.
3. document_reference: The reference number (e.g., "23-6819").
4. title: The COMPLETE AND FULL title including ALL legal language. For ordinances, include "An Ordinance of the City Commission" at the beginning. For resolutions, include "A Resolution of the City Commission" at the beginning. DO NOT truncate or shorten ANY part of the legal title. CRITICAL: Extract the COMPLETE item description including all words after 'declaring', 'authorizing', 'approving', 'providing', 'establishing', 'amending', etc. Many agenda items have incomplete descriptions ending with these words - capture the full sentence and complete thought.
5. sponsors: A list of all sponsors, presenters, or requesters (e.g., ["Commissioner Vince Lago", "Mayor Cason"]).
6. motions: A list of any motions made, including who moved and seconded (e.g., ["Moved by Keon, seconded by Quesada"]).
7. voting_summary: A summary of the vote if available (e.g., "Ayes: Cason, Keon, Lago; Nays: None").
8. has_items: true if the section has items, false if it says "None".

CRITICAL: For legal documents (ordinances/resolutions), preserve the ENTIRE title exactly as written. Do NOT remove "An Ordinance of the City Commission" or "A Resolution of the City Commission" prefixes. NEVER truncate titles that end with incomplete phrases like "declaring", "authorizing", "approving", "providing", "establishing" - these indicate the title continues and you must capture the complete sentence.

Return a JSON array of all extracted sections and items.

Document text:

{CHUNK_TEXT}

""

2) ENHANCED MEETING INFORMATION

System Prompt:

“

You are a JSON extraction assistant. Return ONLY valid JSON, no markdown formatting or code blocks.

”

User Prompt:

“

Extract meeting information from this city commission agenda. Find:

1. Meeting type (Regular, Special, Workshop)
2. Meeting time
3. Meeting location/venue
4. Commission members present (if listed)
5. City officials (Mayor, City Manager, City Attorney, City Clerk)

Return ONLY the JSON object below, no other text:

```
{
  "type": "Regular Meeting",
  "time": "5:30 PM",
  "location": "City Commission Chambers",
  "commissioners": ["Name1", "Name2"],
  "officials": {
    "mayor": "Mayor Name",
    "city_manager": "Manager Name",
    "city_attorney": "Attorney Name",
    "city_clerk": "Clerk Name"
  }
}
```

Text (first 3000 chars):

{TEXT_PREVIEW}

”

3) ENTITY EXTRACTION FROM AGENDA ITEMS:

System Prompt:

"""

You are an expert entity extractor. Return only valid JSON arrays.

"""

User Prompt:

"""

Extract entities from these agenda items.

CRITICAL: Each entity MUST have the correct ID field based on its type:

- Person entities: use "personID"
- Organization entities: use "orgID"
- Location entities: use "locationID"
- Document entities: use "documentID" (NOT docID!)
- Agendaltem entities: use "agendaltemID" (NOT agendaID!)
- Meeting entities: use "meetingID"
- Policy entities: use "policyID"

Return a JSON array where each entity has:

```
{
  "type": "Person|Organization|Location|Document|Agendaltem|Meeting|Policy",
  "name": "Entity Name",
  "personID/orgID/locationID/documentID/agendaltemID/meetingID/policyID": "unique_id_value",
  "description": "Brief description"
}
```

Example:

```
[
  {
    "type": "Document",
    "name": "Ordinance 2024-01",
    "documentID": "document_ord_2024_01",
    "description": "Ordinance regarding zoning"
  },
  {
    "type": "Agendaltem",
    "name": "Item E-1",
    "agendaltemID": "agenda_item_e1_2024_01_09",
    "description": "Discussion of ordinance"
  }
]
```

Agenda items:

{AGENDA_ITEMS_TEXT}

"""

4) ENHANCED DOCUMENT LINKER: AGENDA CODE EXTRACTION

System Prompt:

"""

You are a precise data extractor for {DOC_TYPE} documents. Find and extract only the agenda item code. Search the ENTIRE document thoroughly.

"""

User Prompt:

"""

You are analyzing a City of Coral Gables {DOC_TYPE} document (Document #{DOCUMENT_NUMBER}).

Your task is to find the AGENDA ITEM CODE referenced in this document.

CRITICAL INSTRUCTIONS:

1. Search the ENTIRE document for agenda item references
2. Return ONLY the code in this format: AGENDA_ITEM: [code]
3. The code should be ONLY the letter and number (e.g., E-2, F-10, H-1)
4. Do NOT include any explanations, reasoning, or additional text
5. If no agenda item is found, return: AGENDA_ITEM: NOT_FOUND

Examples of valid responses:

- AGENDA_ITEM: E-2
- AGENDA_ITEM: F-10
- AGENDA_ITEM: H-1
- AGENDA_ITEM: NOT_FOUND

DO NOT RETURN ANYTHING ELSE. NO EXPLANATIONS.

Full document text:

{DOCUMENT_TEXT}

"""

5) ENHANCED DOCUMENT LINKER: LEGAL DOCUMENT VALIDATION

System Prompt:

"""

You are a legal document analyzer specializing in City of Coral Gables {DOC_TYPE} documents. Extract precise voting and passage information.

"""

User Prompt:

"""

You are analyzing a City of Coral Gables {DOC_TYPE} document to extract voting and passage information.

DOCUMENT TEXT (first 3000 characters):

{DOCUMENT_TEXT}

CURRENT EXTRACTION RESULTS:

- Outcome Status: {OUTCOME_STATUS}
- Vote Details: {VOTE_DETAILS}

INSTRUCTIONS:

1. Analyze the document text to determine the ACTUAL status
2. Look for any voting information, reading status, or outcome indicators
3. Provide your analysis in this EXACT JSON format:

```
{
  "passed_first_reading": true/false,
  "passed_second_reading": true/false,
  "outcome_status": "Passed|Failed|First Reading Passed|Second Reading Passed|Deferred|Tabled|Pending",
  "vote_details": {
    "yeas": "comma-separated names or count",
    "nays": "comma-separated names or count",
    "unanimous": true/false,
    "abstentions": "if any"
  },
  "confidence_score": 0.0-1.0,
  "reasoning": "Brief explanation of your analysis"
}
```

CRITICAL RULES:

- Only return valid JSON
- For ordinances, distinguish between first reading and final adoption
- For resolutions, they typically pass in one reading unless stated otherwise
- Look for phrases like "PASSED AND ADOPTED", "unanimous", "first reading", etc.
- If vote details show names, count them for outcome determination
- Confidence score should be 0.9+ if you're very certain, 0.5-0.8 if partially certain, <0.5 if unclear

"""

6) NER TRIPLE EXTRACTION:

System Prompt (customized based on the group):

"""

You are a entity extraction for group [N] types: [entity_types] for city government documents. Extract structured knowledge as triples (subject-predicate-object) that conform to the provided ontology. Return only valid JSON.

"""

User Prompt:

"""

FOCUSED ONTOLOGY FOR GROUP [N]: [GROUP_NAME]

=====

ENTITY TYPES:

[Detailed entity definitions for this group's types from ontology_context_camelCase.txt]

RELATIONSHIPS:

[Relevant relationships where source or target is in this group]

IMPORTANT: For this extraction, focus ONLY on these entity types:

[comma-separated list of group entity types]

Ignore all other entity types for now - they will be extracted separately.

=== TRIPLE EXTRACTION PROMPT ===

DOCUMENT CONTEXT:

- Type: {DOC_TYPE_TITLE}
- Date: {MEETING_DATE}
- Source: {SOURCE_FILE_NAME}

EXTRACTION INSTRUCTIONS:

1. Extract knowledge as triples where:

- Subject: An entity with type, ID, and attributes
- Predicate: A relationship type from the ontology
- Object: Another entity with type, ID, and attributes

2. ID Generation Rules (MANDATORY):

- For Agendaltem and Event: <type_prefix>_<normalized_slug>_<YYYY_MM_DD>
- For Policy entities: Use "policy_ordinance_" or "policy_resolution_" prefix
- For ALL OTHER entities: <type_prefix>_<normalized_slug>
- ALL IDs must be lowercase

3. Include ALL available attributes for each entity based on the ontology

4. Use camelCase for all attribute names

5. Extract both explicit and implicit relationships

6. One entity can appear in multiple triples - use consistent IDs

[Detailed triple examples showing proper format]

Text to analyze:

{CHUNK_TEXT}

Return JSON format:

```
{
  "triples": [
    {
      "subject": { "type": "EntityType", "attributes": {...} },
      "predicate": "relationshipType",
      "object": { "type": "EntityType", "attributes": {...} },
      "provenance": {
        "chunkId": "{CHUNK_ID}",
        "textEvidence": "relevant text snippet"
      }
    }
  ],
  "documentContext": {
    "documentID": "{DOC_ID}",
    "chunkID": "{CHUNK_ID}",
    "meetingDate": "{MEETING_DATE}"
  }
}
```

Example of applied prompt for one ontology group:

- **System Prompt:**

"""

You are an entity extractor for group 1 types: Person, Role, Organization for city government documents. Extract structured knowledge as triples (subject-predicate-object) that conform to the provided ontology. Return only valid JSON.

"""

- **User Prompt:**

"""

FOCUSED ONTOLOGY FOR GROUP 1: PEOPLE & ROLES

=====

ENTITY TYPES:

Person

Definition: An individual involved in or referenced by government activities, such as an official, staff member, applicant, or member of the public.

Attributes: personID, name, title, affiliation, contactInfo

Examples: "Mayor Jane Smith," "Council Member John Doe," "City Manager Alice Johnson"

Role

Definition: The specific function, title, or position held by a Person within an Organization or Event.

Attributes: roleID, title, startDate, endDate

Examples: "Mayor," "Committee Chair," "Sponsor," "Applicant"

Organization

Definition: A formal group, institution, government body, legislative council, department, committee, or external agency.

Attributes: orgID, name, type, jurisdiction, address

Examples: "City Council," "Department of Transportation," "Planning and Zoning Committee"

RELATIONSHIPS:

isMemberOf

From: Person → To: Organization

Links an individual to a group they belong to.

holdsRole

From: Person → To: Role

Assigns a specific function or title to a person.

performsAction

From: Person, Organization → To: Action

Links an agent to a procedural step.

sponsors

From: Person, Organization → To: Policy, Project

Signifies formal sponsorship or primary advocacy.

IMPORTANT: For this extraction, focus ONLY on these entity types:

Person, Role, Organization

Ignore all other entity types for now - they will be extracted separately.

=== TRIPLE EXTRACTION PROMPT ===

DOCUMENT CONTEXT:

- Type: Agenda
- Date: 01.09.2024
- Source: agenda_01_09_2024.json

EXTRACTION INSTRUCTIONS:

1. Extract knowledge as triples where:

- Subject: An entity with type, ID, and attributes
- Predicate: A relationship type from the ontology
- Object: Another entity with type, ID, and attributes

2. ID Generation Rules (MANDATORY):

- For Agendaltem and Event: <type_prefix>_<normalized_slug>_<YYYY_MM_DD>
- For Policy entities: Use "policy_ordinance_" or "policy_resolution_" prefix
- For ALL OTHER entities: <type_prefix>_<normalized_slug>
- ALL IDs must be lowercase

3. Include ALL available attributes for each entity based on the ontology

4. Use camelCase for all attribute names
5. Extract both explicit and implicit relationships
6. One entity can appear in multiple triples - use consistent IDs

TRIPLE EXAMPLES:

From text: "Commissioner Smith moved to approve Ordinance 2024-01"

Extract:

```
{  
  "subject": {  
    "type": "Person",  
    "attributes": {  
      "personID": "person_john_smith",  
      "firstName": "John",  
      "lastName": "Smith",  
      "title": "Commissioner",  
      "affiliation": null,  
      "contactInfo": null  
    }  
  },  
  "predicate": "performsAction",  
  "object": {  
    "type": "Action",  
    "attributes": {  
      "actionID": "action_approve",  
      "type": "approve",  
      "description": "The Commissioner is approving Ordinance 2024-01."  
    }  
  }  
}
```

```
{
  "dateTime": "01.09.2024",
  "outcome": null,
  "details": "Motion to approve Ordinance 2024-01"
}
```

Text to analyze:

CITY COMMISSION REGULAR MEETING

January 9, 2024

PRESENT:

Mayor Vince Lago

Vice Mayor Rhonda Anderson

Commissioner Jorge Fors Jr.

Commissioner Kirk Menendez

Commissioner Ariel Fernandez-Rundle

ALSO PRESENT:

City Manager Peter Iglesias

City Attorney Miriam Ramos

City Clerk Billy Urquia

E-1. RESOLUTION: A RESOLUTION OF THE CITY COMMISSION APPROVING THE APPOINTMENT OF MARIA GONZALEZ TO THE PARKS AND RECREATION ADVISORY BOARD.

Commissioner Fors moved to approve the appointment. Vice Mayor Anderson seconded the motion.

The Parks Department recommends approval of this appointment for a three-year term.

Return JSON format:

```
{
  "triples": [
    {
      "subject": {
        "type": "Person",
        "attributes": {
          "personID": "person_mayor_vince_lago",
          "name": "Vince Lago",
          "title": "Mayor",
          "affiliation": "City Commission",
          "contactInfo": null
        }
      },
      "predicate": "holdsRole",
      "object": {
        "type": "Role",
        "attributes": {
          "roleID": "role_mayor",
          "title": "Mayor",
          "startDate": null,
          "endDate": null
        }
      }
    }
  ]
}
```



```
}  
  
,  
  
"provenance": {  
  "chunkId": "chunk_001",  
  "textEvidence": "Mayor Vince Lago"  
}  
  
,  
  
{  
  "subject": {  
    "type": "Person",  
    "attributes": {  
      "personID": "person_commissioner_jorge_fors_jr",  
      "name": "Jorge Fors Jr.",  
      "title": "Commissioner",  
      "affiliation": "City Commission",  
      "contactInfo": null  
    }  
  },  
  
  "predicate": "performsAction",  
  
  "object": {  
    "type": "Action",  
    "attributes": {  
      "actionID": "action_move_approval",  
      "type": "move",  
      "dateTime": "01.09.2024",  
      "outcome": null,  

```

```
    "details": "Commissioner Fors moved to approve the appointment"
  }
},
"provenance": {
  "chunkId": "chunk_001",
  "textEvidence": "Commissioner Fors moved to approve the appointment"
}
},
{
  "subject": {
    "type": "Organization",
    "attributes": {
      "orgID": "org_parks_department",
      "name": "Parks Department",
      "type": "department",
      "jurisdiction": "City",
      "address": null
    }
  },
  "predicate": "sponsors",
  "object": {
    "type": "Policy",
    "attributes": {
      "policyID": "policy_resolution_appointment_maria_gonzalez",
      "ordinanceNumber": null,
      "title": "Appointment of Maria Gonzalez to Parks Advisory Board",
```

```

    "status": "proposed",

    "effectiveDate": null,

    "expirationDate": null,

    "legalReferences": [],

    "documentRepresentation": null

  },

  "provenance": {

    "chunkId": "chunk_001",

    "textEvidence": "The Parks Department recommends approval"

  }

},

],

"documentContext": {

  "documentID": "document_agenda_01_09_2024",

  "chunkID": "chunk_001",

  "meetingDate": "01.09.2024"

}

}

""

```

7) QUERY CLASSIFICATION

System Prompt:

```

""

```

You classify queries and extract entities. Return only valid JSON.

""

User Prompt:

""

Analyze this query and classify it into one of these categories:

QUERY TYPES:

- **specific_fact**: Simple lookup for ONE specific entity or count (WHO IS person X, WHAT IS ordinance Y, HOW MANY items)
- **general_info**: Open-ended questions about broad topics, policies, or procedures
- **complex_hybrid**: Questions requiring MULTIPLE data sources or connections between different entity types
- **unclear**: Ambiguous queries needing clarification

CRITICAL CLASSIFICATION RULES:

- "Who is [person name]?" = **specific_fact** (single person lookup)
- "What is ordinance 2024-01?" = **specific_fact** (single ordinance lookup)
- "How many [items]?" = **specific_fact** (simple count)
- "Tell me about [general topic]" = **general_info** (broad thematic information)
- "What policies cover [topic]?" = **general_info** (broad conceptual question)

COMPLEX_HYBRID INDICATORS (use this when query has ANY of these):

- Asks about connections between different entities ("security measures discussed in meetings about ordinances")
- Requires finding entities AND their related content ("which ordinances had public opposition")
- Involves multiple entity types that need to be connected ("commissioners who voted on parking-related resolutions")

- Asks "what...discussed in meetings about..." (needs meeting content + specific topics)
- Contains words: "relationship", "connections", "discussed in", "mentioned in", "related to"

ENTITY TYPES TO EXTRACT:

- Person: Officials, commissioners, mayors, staff members
- Organization: Departments, committees, boards, companies
- Document: ANY mention of documents including:
 - * Ordinances (legislative documents)
 - * Resolutions (legislative documents)
 - * Agendas (meeting planning documents)
 - * Minutes (meeting record documents)
 - * Verbatim transcripts (word-for-word meeting records)
 - * Reports, memos, proclamations
 - * Any document type reference
- Policy: Specific policies, rules, or regulations (but NOT ordinances/resolutions - those are Documents)
- Event: General events, hearings, workshops (use Meeting for specific city meetings)
- Meeting: Formal city government meetings (City Council, committee meetings)
- Action: Votes, approvals, motions
- Asset: Funds, budgets, monetary amounts
- Project: Initiatives, developments, programs
- Location: Addresses, buildings, districts
- Role: Positions, titles, functions
- Topic: Subjects, issues, themes
- Section: Agenda sections (like "Consent Agenda", "Regular Business")
- AgendaItem: Items like E-1, F-10, agenda references
- AgendaDocument: Complete agenda documents for meetings

- Contract: Agreements, RFPs, bids
- Technology: Systems, software, platforms
- VoteOutcome: Voting results, decisions
- Presentation: Presentations given during meetings
- PublicComment: Comments from members of the public
- Board: Boards and committees (when referring to the body itself)
- Appointment: Appointments to boards, positions, or roles
- LegalReference: Citations to laws, codes, or regulations

IMPORTANT RECOGNITION RULES:

- "ordinances" → Document (subtype: ordinance)
- "resolutions" → Document (subtype: resolution)
- "agenda" or "agendas" → Document (subtype: agenda) or AgendaDocument
- "minutes" → Document (subtype: minutes)
- "verbatim" or "transcript" or "verbatim transcript" → Document (subtype: verbatim)
- "report" → Document (subtype: report)
- "consent agenda", "regular business" → Section
- "city council meeting", "committee meeting" → Meeting
- "presentation" → Presentation
- "public comment" → PublicComment
- "board", "committee" (as entity) → Board
- "appointment" → Appointment
- "section", "chapter", "code" (legal) → LegalReference
- Any plural or singular form should be recognized

Query: "{USER_QUERY}"

Return JSON:

```
{
  "query_type": "specific_fact|general_info|complex_hybrid|unclear",
  "confidence": 0.0-1.0,
  "entities": [
    {
      "type": "Entity type from list above",
      "value": "Exact text from query",
      "normalized": "normalized form (lowercase, no titles)",
      "subtype": "for Document entities only - the document subtype"
    }
  ],
  "reasoning": "Brief explanation of classification"
}
```

Examples:

- "How many ordinances were passed in January 2024?"

→ specific_fact, entities: [{"type": "Document", "value": "ordinances", "subtype": "ordinance"}, {"type": "Event", "value": "January 2024"}]

- "Show me all resolutions about parking"

→ specific_fact, entities: [{"type": "Document", "value": "resolutions", "subtype": "resolution"}, {"type": "Topic", "value": "parking"}]

- "What was discussed in the verbatim transcript?"

→ general_info, entities: [{"type": "Document", "value": "verbatim transcript", "subtype": "verbatim"}]

"""

8) GRAPH QUERY GENERATION:

System Prompt:

"""

You are an expert at generating Gremlin queries for Azure Cosmos DB. Return only valid JSON.

"""

User Prompt:

"""

Generate a Gremlin query for Azure Cosmos DB based on this request.

USER QUERY: "{ORIGINAL_QUERY}"

QUERY TYPE: {QUERY_TYPE}

EXTRACTED ENTITIES: {ENTITIES_JSON}

GRAPH SCHEMA:

{SCHEMA_INFO}

IMPORTANT JSON FORMAT:

- Return ONLY valid JSON with "query" and "explanation" fields
- Do NOT use string concatenation (+ operators) in JSON

- Do NOT use line breaks or escape characters in the query string
- Write the full query as one complete single-line string
- Example: {"query": "g.V().hasLabel('person').valueMap(true)", "explanation": "Get person info"}
- WRONG: Query with \n line breaks or \ escapes
- CORRECT: Single line query string without escapes

CRITICAL RULES - NEVER HARDCODE DATES:

1. NEVER assume a specific year unless explicitly mentioned in the query
2. "last meeting" = order by date descending, take first (NO date filter)
3. "recent" = order by date descending, limit to reasonable number (NO date filter)
4. "all" = no date filter at all
5. Only filter by date if the user explicitly mentions a date/year
6. The system contains data from 2015-2025 - work across all years

COSMOS DB SYNTAX:

- Use 'decr' for descending order (not 'desc')
- Use 'incr' for ascending order (not 'asc')
- Event date property is called 'dateTime' (for events/meetings)
- Agenda item date property is called 'meetingDate' (for agenda items)
- Document date property is called 'meetingDate' (for documents)
- Ordinance/Policy date property is called 'meetingDate' (for ordinances)
- Always use valueMap(true) to get actual property values
- Entity labels match ontology (lowercase): event, agendadocument, section, agendaitem, policy, document
- CRITICAL: Ordinances are stored as hasLabel('policy') with ordinanceNumber field
- NEVER USE: policyType='ordinance' (this field does NOT exist in the database)
- FOR ORDINANCES: Use has('ordinanceNumber') to identify ordinance entities

- CRITICAL: Gremlin traversal syntax - NO underscores in method names:

- * Use `.in('relationshipName')` NOT `.in_('relationshipName')`
- * Use `.out('relationshipName')` NOT `.out_('relationshipName')`
- * Use `.count()` NOT `.count_()`
- * Use `.as('label')` NOT `.as_('label')`
- * Use `.select('label')` NOT `.select_('label')`

- CRITICAL: Documents vs Agenda Items are DIFFERENT:

- * Documents = `hasLabel('document')` - actual files (PDFs, transcripts, reports)
- * Agenda Items = `hasLabel('agendaitem')` - items that appear on meeting agendas

- CRITICAL: For document queries with dates, ALWAYS filter for source documents:

- * Add `.has('sourceFileName')` to ensure it's a file-based document (helps avoid fragments)
- * This filters for documents that represent actual files rather than extracted content
- * NEVER use event/meeting traversals when user asks for "documents" - always use direct `hasLabel('document')` queries

- Correct relationship path for agenda items: `event` → `hasAgenda` → `agendadocument` → `hasSection` → `section` → `hasAgendaitem` → `agendaitem`

- For simple entity queries, use direct `hasLabel()` without traversals

CRITICAL DATE FORMATS:

- Dates are stored as MM.DD.YYYY format (e.g., '01.09.2024' for January 9, 2024)
- When user mentions dates like "January 9 2024", convert to '01.09.2024'
- When user mentions dates like "2024-01-09", convert to '01.09.2024'
- When user mentions dates like "01-09-2024", convert to '01.09.2024'
- Use `'containing()'` for exact date searches

CRITICAL TEMPORAL QUERIES:

- "since YEAR" = from beginning of that year onwards (e.g., "since 2010" = from 01.01.2010 onwards)
- "before YEAR" = up to end of previous year (e.g., "before 2020" = up to 12.31.2019)
- "after YEAR" = from beginning of next year onwards (e.g., "after 2015" = from 01.01.2016 onwards)
- "from YEAR1 to YEAR2" = from beginning of YEAR1 to end of YEAR2
- For date ranges, use multiple .or() conditions or .has() filters for different years
- NEVER use containing() for range queries - use year-by-year filtering

CRITICAL PERSON NAME MATCHING:

- Person names are stored in separate firstName and lastName fields (e.g., firstName: 'Vince', lastName: 'Lago')
- User queries can be in any case ('vince lago', 'VINCE LAGO', 'Vince Lago')
- Handle reversed name order by checking both firstName/lastName combinations
- Use case-insensitive matching with .or() conditions
- Check firstName, lastName, and title fields when searching for people
- For name queries: firstName + lastName, lastName + firstName (reversed), individual names, title matches

EXAMPLES OF CORRECT QUERIES:

[Extensive examples section continues...]

”

9) RESPONSE SYNTHESIS

System Prompt:

”

You synthesize information from multiple sources into clear, structured answers.

”

User Prompt:

Synthesize a comprehensive answer from the following data sources.

USER QUERY: "{USER_QUERY}"

GRAPH DATA:

{FORMATTED_GRAPH_RESULTS}

VECTOR DATA:

{FORMATTED_VECTOR_RESULTS}

INSTRUCTIONS:

1. Structure your answer with an EXECUTIVE SUMMARY at the top, followed by detailed bullet points
2. Use this exact format:

Executive Summary

[Brief 2-3 sentence overview of the key findings]

Detailed Breakdown

[Category 1 Name]

- [Item 1] [Citation]
- [Item 2] [Citation]
- [Item 3] [Citation]

[Category 2 Name]

- [Item 1] [Citation]
- [Item 2] [Citation]

3. Use inline citations like [G1] for graph data or [V1] for text data after each bullet point

4. Group related items into logical categories (e.g., "Awards & Recognition", "Appointments", "Ordinances", "Updates", etc.)

5. Be specific with names, dates, and numbers from the data

6. Keep bullet points concise but informative

7. If data sources contradict, mention both perspectives

8. CRITICAL FOR AGENDA ITEMS: Extract the item code from entries like 'Agenda Item A-1: [title]' and format bullet points as '[CODE]: • [description] [Citation]' (e.g., 'A-2: • Recognition of Coral Gables Fire Department Rescue 4 Crew [G1]')

9. CONSISTENT ATTRIBUTE FORMATTING: For all entity types, use consistent formatting patterns:

- For Ordinances: Always use "Ordinance [NUMBER]:" format (e.g., "Ordinance 2024-02:" or "Ordinance 2551:")
- For Resolutions: Always use "Resolution [NUMBER]:" format
- For Policies: Always use "Policy [IDENTIFIER]:" format if identifier exists
- For Appointments: Always use "Appointment of [NAME] to [BOARD]:" format
- For all entities: If presenting lists, ensure the same primary attribute is shown for each item of the same type

Write the synthesized answer using the structured format above:

”

10) MULTI-HOP SYNTHESIS

System Prompt:

”

You synthesize information from multiple sources into clear answers.

""

User Prompt:

""

Synthesize a complete answer from multiple data sources.

ORIGINAL QUERY: "{ORIGINAL_QUERY}"

HOP SEQUENCE:

{HOP_SEQUENCE}

ALL RESULTS:

{ALL_RESULTS}

Create a comprehensive answer that:

1. Directly addresses the original query
2. Combines information from all hops
3. Cites which hop provided which information
4. Is clear and well-structured

Answer:

""

ONTOLOGY:

Final Comprehensive Ontology for City Governance

This ontology provides a definitive, unified schema for structuring data from diverse city government documents. By integrating detailed feedback, this model adopts an action-centric approach, enhancing its capacity for procedural and temporal analysis.



Entity Types

These are the primary classes of objects, or "nouns," in the city governance domain.



Person

- **Definition:** An individual involved in or referenced by government activities, such as an official, staff member, applicant, or member of the public.
- **Examples:** "Mayor Jane Smith," "Council Member John Doe," "City Manager Alice Johnson," "Commissioner Smith," "Smith"
- **Attributes:**
 - **personID:** A unique identifier for the individual.
 - **firstName:** The person's first name (e.g., "Vince", "Jane", "John").
 - **lastName:** The person's last name (e.g., "Lago", "Smith", "Doe").
 - **title:** The official title (e.g., "The Honorable," "Dr.", "Mayor", "Commissioner").
 - **affiliation:** The primary Organization the person is associated with.
 - **contactInfo:** Email, phone number, or other contact details.



Organization

- **Definition:** A formal group, institution, government body, legislative council, department, committee, or external agency.
- **Examples:** "City Council," "Department of Transportation," "Planning and Zoning Committee," "Planning Department," "ABC Corporation"
- **Attributes:**
 - **orgID:** A unique identifier for the organization.
 - **name:** The official name of the organization.
 - **type:** The category of the organization (e.g., "Department," "Committee," "Agency," "Legislative Body").
 - **jurisdiction:** The geographical or administrative area of authority.
 - **address:** The primary physical address of the organization.



Document

- **Definition:** An official record, report, piece of correspondence, or meeting minutes. This is the central source of information and evidence.
- **Examples:** "Meeting Minutes of 09-10-2025," "Staff Report on Traffic," "Annual Budget Report," "Staff Report SR-2024-123," "Meeting Minutes 01-09-2024," "Public Hearing Notice PH-2024-45"
- **Attributes:**
 - **documentID:** A unique identifier for the document.
 - **title:** The official title of the document.
 - **type:** The category of the document (e.g., "Minutes," "Report," "Memo").
 - **status:** The current state (e.g., "Draft," "Final," "Archived").
 - **issueDate:** The date the document was published or filed.
 - **version:** A version number or revision date.
 - **summary:** A brief description of the document's content.
 - **sourceURL:** A link to the original document file.
 - **hyperlinks:** Array of {url, text, page} objects for links within the document.
 - **meetingDate:** The date of the associated meeting (for meeting-related documents).
 - **documentClassification:** Standardized classification category.
 - **sourceFileName:** Original filename of the document.
 - **sourceFilePath:** Full path to the source file.

Policy

- **Definition:** A formal rule, law, ordinance, resolution, or regulation that is proposed, enacted, or referenced. It represents the legal/regulatory content. For ordinances and regulations, a separate **Document** node should also be created to represent the physical document.
- **Examples:** "Ordinance 2025-12," "Resolution R-2025-45," "City Zoning Code Section 5.1," "Ord. 2024-01," "Res. R-23-456," "Emergency Ordinance E-2024-12," "Ordinance 2023-45," "Resolution R-22-123"
- **Attributes:**
 - **policyID:** A unique identifier for the policy (e.g., "policy_ordinance_2024_01").
 - **ordinanceNumber:** The official ordinance number if applicable.
 - **title:** The official title of the legislation.
 - **status:** The current legislative state (e.g., "Proposed," "Enacted," "Repealed," "Amended").
 - **effectiveDate:** The date the policy's rules come into force.
 - **expirationDate:** The date the policy is no longer in effect.
 - **legalReferences:** An array of citations to other laws or codes.
 - **documentRepresentation:** Reference to the associated Document node (for ordinances/regulations).

Event

- **Definition:** A specific, planned occurrence in time, such as a meeting, public hearing, vote, or workshop.
- **Examples:** "City Commission Regular Meeting," "Public Hearing on Budget," "Public Hearing on January 23, 2024"

- **Attributes:**

- **eventID:** A unique identifier for the event.
- **name:** The official name or title of the event.
- **type:** The category of the event (e.g., "Regular Meeting," "Public Hearing," "Workshop").
- **dateTime:** The specific date and time the event occurs.
- **status:** The event's state (e.g., "Scheduled," "Completed," "Canceled").
- **outcome:** The result of the event (e.g., "Adjourned," "Motions Passed").
- **docId:** Document identifier associated with the event.
- **sourceFileName:** Name of the source file containing event information.

Action

- **Definition:** A specific, discrete procedural step or activity performed by a **Person** or **Organization**. This entity makes processes explicit.
- **Examples:** "Vote on Ordinance 2025-12," "Approval of contract," "Amendment to Zoning Code," "approved," "deferred," "amended"
- **Attributes:**
 - **actionID:** A unique identifier for the action instance.
 - **type:** The category of the action (e.g., "Vote," "Approval," "Proposal," "Amendment," "Repeal").
 - **dateTime:** The timestamp when the action was performed.
 - **outcome:** The result of the action (e.g., "Passed 5-2," "Approved," "Denied").
 - **details:** A textual description of the action (e.g., "Motion by Smith, second by Doe").

Asset

- **Definition:** A physical, financial, or other resource of value to the city. This replaces **Financial Entity** to be more inclusive.
- **Examples:** "2025 Parks Budget," "Fleet Vehicle 12," "Central Park Property," "State Infrastructure Grant," "\$150,000 Parks Improvement Fund," "\$2.5 million Infrastructure Bond," "Contract value: \$75,000"
- **Attributes:**
 - **assetID:** A unique identifier (e.g., Parcel Number, Account Number).
 - **name:** A human-readable name for the asset.
 - **type:** The kind of asset (e.g., "Fund," "Grant," "Property," "Vehicle").
 - **value:** A financial or assessed value.
 - **currency:** The currency used (e.g., "USD").
 - **status:** The current state (e.g., "In Service," "Under Construction," "Depleted").
 - **fiscalYear:** The fiscal year associated with a financial asset.

Project

- **Definition:** A specific, planned initiative or undertaking by the city, often with a defined scope, budget, and timeline.

- **Examples:** "Riverside Greenway Development," "Downtown Revitalization Initiative," "Main Street Repaving"
- **Attributes:**
 - **projectID:** A unique identifier for the project.
 - **name:** The common name of the initiative.
 - **description:** A brief summary of the project's goals.
 - **status:** The current state (e.g., "Planned," "Ongoing," "Completed").
 - **startDate:** The official start date of the project.
 - **endDate:** The official or projected end date of the project.

Location

- **Definition:** A physical place, geographical area, or administrative district relevant to government actions.
- **Examples:** "City Hall Commission Chambers," "District 5," "123 Main Street," "405 Biltmore Way, Coral Gables, FL 33134," "Miracle Mile between Ponce de Leon and LeJeune," "City Commission Chambers, 3rd Floor," "City Hall," "Miracle Mile," "33134"
- **Attributes:**
 - **locationID:** A unique identifier for the location.
 - **name:** The common name of the location.
 - **type:** The category of the location (e.g., "Building," "District," "Property").
 - **address:** The full street address.
 - **coordinates:** Geographic latitude and longitude (e.g., 25.8202, -80.1226).

Role

- **Definition:** The specific function, title, or position held by a **Person** within an **Organization** or **Event**.
- **Examples:** "Mayor," "Committee Chair," "Sponsor," "Applicant," "Voter"
- **Attributes:**
 - **roleID:** A unique identifier for the role title.
 - **title:** The name of the role (e.g., "Chairperson").
 - **startDate:** The date the role assignment began.
 - **endDate:** The date the role assignment ended.

Topic

- **Definition:** The subject matter, issue, or abstract concept being discussed or addressed.
- **Examples:** "Affordable Housing," "Traffic Congestion," "Public Safety," "Zoning"
- **Attributes:**
 - **topicID:** A unique identifier for the topic.
 - **name:** The name of the topic.
 - **category:** A broader classification for the topic (e.g., "Transportation," "Housing").
 - **description:** A brief explanation of the topic.

Meeting

- **Definition:** A formal gathering of government officials, typically a city council meeting or committee meeting.
- **Examples:** "City Council Regular Meeting 2024-01-09," "Planning Committee Meeting 2024-02-15"
- **Attributes:**
 - **meetingID:** A unique identifier for the meeting.
 - **date:** The date of the meeting.
 - **docId:** Document identifier associated with the meeting.
 - **sourceFileName:** Name of the source file containing meeting information.
 - **sourceURL:** URL link to the meeting document or video.

Section

- **Definition:** A section or category within an agenda document that groups related agenda items.
- **Examples:** "Regular Business," "Public Hearings," "Consent Agenda," "New Business"
- **Attributes:**
 - **sectionID:** A unique identifier for the section.
 - **code:** The section name or code (e.g., "E", "F", "G").
 - **order:** The order of this section within the agenda.
 - **meetingDate:** The date of the associated meeting.
 - **parentAgendaDocId:** ID of the parent agenda document.
 - **sectionType:** Standardized type (PRESENTATIONS, CONSENT, PUBLIC_COMMENT, REGULAR_BUSINESS, NEW_BUSINESS, EMERGENCY).

AgendaItem

- **Definition:** A specific item on a meeting agenda, typically with an alphanumeric identifier.
- **Examples:** "E-1," "F-10," "R-2024-123"
- **Attributes:**
 - **itemID:** The agenda item identifier.
 - **title:** Brief description of the item.
 - **type:** Category (e.g., "Resolution," "Ordinance," "Discussion").
 - **presenter:** Person presenting the item.
 - **estimatedDuration:** Expected discussion time.
 - **meetingDate:** The date of the meeting this item belongs to.
 - **documentReference:** Reference to associated document (e.g., "Ordinance 2024-01").
 - **order:** Numeric order within the section.
 - **documentType:** Type of document being discussed.
 - **documentClassification:** Classification of the associated document.
 - **parentSectionId:** ID of the parent Section.
 - **sourceURLs:** Array of related URLs.
 - **hyperlinks:** Array of {url, text, page} objects for richer link metadata.

AgendaDocument

- **Definition:** The formal agenda document for a specific meeting, containing all agenda sections and items. This is a specialized **Document** that structures the meeting's business.
- **Examples:** "Agenda for City Council Meeting 2024-01-09", "Planning Committee Agenda 2024-02-15"
- **Attributes:**
 - **agendaDocID:** A unique identifier for the agenda document.
 - **title:** The title of the agenda document.
 - **type:** Fixed as "agenda".
 - **status:** The current state (e.g., "Draft", "Final").
 - **issueDate:** The date the agenda was published.
 - **meetingDate:** The date of the associated meeting.
 - **parentMeetingId:** ID of the associated **Event** (meeting).
 - **Source_File_Name:** Name of the source agenda file.
 - **Source_File_Path:** Full path to the source agenda file.
 - **sourceURL:** URL link to the agenda document.

Contract

- **Definition:** A formal agreement between the city and another party.
- **Examples:** "Contract No. 2024-15," "RFP-2023-456," "Bid Award #789," "Contract No. 2024-15"
- **Attributes:**
 - **contractID:** Unique contract identifier.
 - **title:** Contract description.
 - **vendor:** Organization awarded the contract.
 - **amount:** Total contract value.
 - **startDate:** Contract effective date.
 - **endDate:** Contract expiration date.
 - **status:** Current state (e.g., "Active," "Pending," "Expired").

Technology

- **Definition:** Software, platform, or technical system used by the city.
- **Examples:** "Microsoft Teams," "Granicus," "Accela," "Tyler Munis"
- **Attributes:**
 - **techID:** Unique identifier.
 - **name:** Product name.
 - **vendor:** Company providing the technology.
 - **purpose:** Primary use case.
 - **licenseType:** Licensing model.

VoteOutcome

- **Definition:** Detailed record of a voting action with specific tallies.

- **Examples:**
 - `{"id": "outcome_E-1_2024-01-09", "type": "vote_outcome", "status": "passed", "yes_votes": 5, "no_votes": 2, "details": "Motion passed with Commissioner X abstaining"}`
 - "Vote on Ord. 2024-01: Passed 5-2"
 - "Motion to defer E-1: Failed 3-4"
- **Attributes:**
 - `outcomeID`: Unique identifier (e.g., "outcome_E-1_2024-01-09").
 - `agendaItemID`: Related agenda item (was "agendaItem").
 - `result`: The outcome (alternative to "status").
 - `status`: Result (e.g., "passed", "failed", "tabled", "deferred").
 - `yesVotes`: Number of affirmative votes.
 - `noVotes`: Number of negative votes.
 - `abstentions`: Number of abstentions.
 - `voteDetails`: Array of [Person, vote] pairs.

Relationship Types

These connections define how entities interact. This model is action-centric, using the `Action` entity to explicitly represent processes.

Relationship	Connects (Source → Target)	Description & Example
<code>isMemberOf</code>	<code>Person</code> → <code>Organization</code>	Links an individual to a group they belong to. Ex: "Jane Smith" <code>isMemberOf</code> "City Council."
<code>isPartOf</code>	<code>Organization</code> → <code>Organization</code>	Defines institutional hierarchy. Ex: "Planning Committee" <code>isPartOf</code> "Department of Development."
<code>holdsRole</code>	<code>Person</code> → <code>Role</code>	Assigns a specific function or title to a person. Ex: "Jane Smith" <code>holdsRole</code> "Mayor."

participatesIn	Person, Organization → Event	Shows attendance or active involvement in an event. Ex: "Mayor Doe" participatesIn "Budget Hearing."
authoredBy	Document, Policy → Person, Organization	Identifies the creator or official issuer of a document. Ex: "Staff Report" authoredBy "Dept. of Transport."
sponsors	Person, Organization → Policy, Project	Signifies formal sponsorship or primary advocacy for an initiative. Ex: "Council Member Doe" sponsors "Resolution R-2025-45."
performsAction	Person, Organization → Action	The core relationship linking an agent to a procedural step. Ex: "City Council" performsAction "Action #789" (Vote on Ordinance).
targetOf	Action → Document, Policy, Project, Asset	Connects an Action to the entity it affects. Ex: "Action #789" is targetOf "Ordinance 2025-12."
recordedIn	Action → Document (EXPANDED)	Indicates where an action or event is officially recorded. Ex: "Action #789" recordedIn "Meeting Minutes". Ex: "City Council Meeting" recordedIn "Verbatim Transcript 2024-01-09"
isLocatedAt	Organization, Project → Location	Specifies the primary, static physical location of an entity. Ex: "The Dept. of Parks" isLocatedAt "City Hall Annex."

occursAt	Event → Location	Specifies the location where a dynamic event takes place. Ex: "The Public Hearing" occursAt "City Hall Council Chambers."
references	Document, Policy → Document, Policy, Topic	Indicates that one document cites or mentions another entity. Ex: "Ordinance 2025-12" references "City Charter §5.1."
amends	Policy → Policy	(Separated) Shows that one legal instrument officially modifies another. Ex: "Ordinance 2025-45" amends "Ordinance 2015-23."
repeals	Policy → Policy	(Separated) Shows that one legal instrument officially nullifies another. Ex: "Ordinance 2025-99" repeals "Ordinance 2011-05."
owns	Person, Organization → Asset	Denotes legal ownership or stewardship of an asset. Ex: "The City" owns "Central Park Property."
funds	Asset → Project, Organization	Specifies the source of funding (Asset of type 'Financial') for an initiative. Ex: "State Grant" funds "Park Renovation Project."
addressesTopic	Document, Event, Project → Topic	Connects a resource or discussion to its primary subject matter. Ex: "Meeting on Housing" addressesTopic "Affordable Housing."
discusses	Event → AgendaItem, Policy, Document, Topic	Links an event to specific agenda items covered. An event can discuss multiple types of entities. Ex: "City Commission Meeting" discusses "E-1."

hasAgenda	Event → AgendaDocument	Links a meeting event to its formal agenda document. Ex: "City Council Meeting 2024-01-09" hasAgenda "Agenda Document for 2024-01-09."
hasSection	AgendaDocument → Section	Links an agenda document to its constituent sections. Ex: "Agenda Document" hasSection "Regular Business Section."
hasAgendaItem	Section → AgendaItem	Links a section to the specific agenda items it contains. Ex: "Regular Business Section" hasAgendaItem "E-1."
precedes	AgendaItem → AgendaItem	Shows the temporal order of agenda items within a meeting. Ex: "E-1" precedes "E-2."
resultsIn	AgendaItem → VoteOutcome	Connects an agenda item to its voting result. Ex: "E-1" resultsIn "outcome_E-1_2024-01-09."
governedBy	Contract → Policy	Links a contract to its authorizing legislation. Ex: "Contract No. 2024-15" governedBy "Ordinance 2024-01."
uses	Organization → Technology	Shows technology adoption by an organization. Ex: "City Clerk's Office" uses "Granicus."
votedOn	VoteOutcome → Policy, Contract, Project	Links a vote outcome to what was voted on. Ex: "outcome_E-1_2024-01-09" votedOn "Ordinance 2024-01."

presents	Person → AgendaItem	Shows who presents an agenda item. Ex: "Commissioner Jane Doe" presents "E-1."
awards	Organization → Contract	Shows which organization awards a contract. Ex: "City Council" awards "Contract No. 2024-15."
awardedTo	Contract → Organization	Shows the recipient of a contract. Ex: "Contract No. 2024-15" awardedTo "ABC Corporation."
implementedBy	Policy → Document	A Policy is implementedBy its Document representation. Links a legal rule to its physical document. Ex: "Ordinance 2024-01" implementedBy "Document: Ordinance 2024-01 PDF."
embodies	Document → Policy	Links a legal document to the policy/rule it contains. Ex: "Document: Ordinance 2024-01 PDF" embodies "Ordinance 2024-01."
implements	AgendaItem → Policy, Document	An agenda item enacts a policy or document. Ex: "Agenda Item E-1" implements "Ordinance 2024-01."
hasTranscript	AgendaItem, Event → Document	Links an agenda item or event to its verbatim transcript document. Ex: "City Council Meeting 2024-01-09" hasTranscript "Verbatim Transcript 2024-01-09"
discussedIn	Document, Policy, AgendaItem → Event	The passive form of "discusses". Ex: "Ordinance 2024-01" discussedIn "City Council Meeting 2024-01-09"

mentionedIn	Any Entity Type → Document	Indicates that an entity is mentioned in or was extracted from a document. Ex: "person_commissioner_smith" mentionedIn "document_agenda_2024_01_09"
isPartOf	AgendaItem → Document , AgendaDocument	Indicates that an agenda item is part of a document. Ex: "agenda_item_e1" isPartOf "document_agenda_2024_01_09"
containsItem	Document , Section → AgendaItem	Indicates containment of agenda items within documents or sections. Ex: "Agenda Document 2024-01-09" containsItem "E-1"

Export to Sheets

AgendaDocument Relationships

AgendaDocument inherits all **Document** relationships PLUS:

- **authoredBy:** **AgendaDocument** → **Person, Organization**
 - Ex: "Agenda Document 2024-01-09" **authoredBy** "City Clerk's Office."
- **references:** **AgendaDocument** → **Document, Policy, Topic**
 - Ex: "Agenda Document" **references** "Budget Policy 2024."
- **recordedIn:** **Action** → **AgendaDocument**
 - Ex: "Meeting Action #123" **recordedIn** "Agenda Document 2024-01-09."
- **hasAgenda:** **Event** → **AgendaDocument**
 - Ex: "City Council Meeting 2024-01-09" **hasAgenda** "Agenda Document for 2024-01-09."
- **hasSection:** **AgendaDocument** → **Section**
 - Ex: "Agenda Document" **hasSection** "Regular Business Section."

Section Relationships

- **hasSection:** **AgendaDocument** → **Section**
 - Ex: "Agenda Document" **hasSection** "Consent Agenda Section."
- **hasAgendaItem:** **Section** → **AgendaItem**

- Ex: "Regular Business Section" **hasAgendaItem** "E-1."
 - **addressesTopic**: **Section** → **Topic**
 - Ex: "Budget Section" **addressesTopic** "Municipal Finance."
 - **precedesSection**: **Section** → **Section**
 - Ex: "Consent Agenda" **precedesSection** "Regular Business."
-

Relationship Direction Guidelines

Some relationships have both active and passive forms:

- **implements** (active) vs. **implementedBy** (passive)
- **authors** (active) vs. **authoredBy** (passive)
- **awards** (active) vs. **awardedTo** (passive)

When both forms exist:

- Use the form that makes the query most natural
- The system should understand both directions
- Storage may use a canonical direction and infer the inverse

For agenda item voting flows:

- **AgendaItem** → **resultsIn** → **VoteOutcome**
- **VoteOutcome** → **votedOn** → **Policy/Contract/Project**

For document relationships:

- **Document** → **embodies** → **Policy** (document contains the policy text)
 - **Policy** → **implementedBy** → **Document** (policy is implemented by the document)
 - **AgendaItem** → **implements** → **Policy** (agenda item enacts the policy)
-

Attribute Naming Conventions

For consistency across the system:

- Use **camelCase** for all attributes (e.g., **meetingDate**, not **meeting_date**).
- ID fields should be **{entityType}ID** (e.g., **personID**, **documentID**).
- Date fields should include "Date" or "DateTime" suffix.
- Status fields should be named "status" (not "state" or "result").
- Source tracking: **Source_File_Name**, **Source_File_Path** (exceptions to camelCase for legacy compatibility).

Appendix — Pipeline Additions (2025-08)

Purpose: capture attributes used by the ingestion/NER/taxonomy code that may not be present in this file yet. These entries are additive; they do not deprecate any existing attributes.

Document (Document)

ID field: `documentID` New/Added Attributes (definitions):

- `documentType` (*string*) — subtype of document. Examples: `agenda`, `ordinance`, `resolution`, `verbatim_transcript`, `transcript`.
- `issueDate` (*date/string*) — document's publication/adoption date.
- `meetingDate` (*date/string*) — meeting date the document belongs to.
- `status` (*string*) — e.g., `Draft`, `Final`.
- `sourceURL` (*string/URL*) — public URL of the underlying file.
- `source_file_name` (*string*) — original file name. (Alias: `Source_File_Name`).
- `source_file_path` (*string*) — original file path. (Alias: `Source_File_Path`).
- `name` (*string, optional*) — human label when distinct from `title`.
- `parentEventId` (*string, optional*) — event this document is attached to. (Alias: `parentMeetingId`).

Policy (Policy)

ID field: `policyID` New/Added Attributes:

- `ordinanceNumber` (*string*) — ordinance identifier, e.g., `2024-03`.
- `resolutionNumber` (*string*) — resolution identifier, e.g., `2024-15`.
- `effectiveDate` (*date/string*) — date the policy takes effect.
- `expirationDate` (*date/string, optional*) — expiry when applicable.
- `legalReferences` (*array*) — citations/sections referenced by the policy.
- `meetingDate` (*date/string*) — meeting date of adoption.
- `status` (*string*) — e.g., `enacted`, `adopted`, `proposed`.
- `parentEventId` (*string, optional*) — event that adopted the policy. (Alias: `parentMeetingId`).

Agenda Item (AgendaItem)

ID field: `agendaItemID` New/Added Attributes:

- `itemID` (*string*) — human code from agenda (e.g., `E-4`).
- `code` (*string*) — same value as `itemID`; kept for compatibility.
- `title` (*string*) — item title/subject.

- **documentReference** (*string*) — reference to associated document (e.g., "23-6764").
- **url** (*string, optional*) — primary URL link for the agenda item.
- **hyperlinks** (*array, optional*) — array of hyperlink objects with full metadata.
- **meetingDate** (*date/string*) — meeting date the item belongs to.
- **subtype** (*string, optional*) — local category, e.g., **Consent**, **Presentation**.
- **presenter** (*string, optional*) — person/office presenting.
- **estimatedDuration** (*string/number, optional*) — expected duration.
- **relativeSectionId** (*string, optional*) — section node this item belongs to.
- **section** (*string, optional*) — section name.

Event (Event)

ID field: **eventID** New/Added Attributes:

- **name** (*string*) — e.g., "City Commission Meeting".
- **meetingDate** (*date/string*) — meeting date.
- **dateTime** (*date/time string*) — date/time as captured from sources.
- **status** (*string*) — lifecycle (e.g., **Completed**).
- **outcome** (*string, optional*) — e.g., **Adjourned**.

Topic (Topic)

ID field: **topicID** New/Added Attributes:

- **category** (*string*) — e.g., **Meeting Section**, **Governance**.
- **description** (*string, optional*) — short description.

Section (Section)

ID field: **sectionID** New/Added Attributes:

- **name** (*string*) — section title, e.g., **CONSENT AGENDA**.
- **meetingDate** (*date/string*) — meeting date.

Person (Person)

ID field: **personID** New/Added Attributes:

- **title** (*string, optional*) — e.g., **Mayor**, **Commissioner**.
- **role** (*string, optional*) — context role for the discussion.
- **affiliation** (*string, optional*) — e.g., **City Council**.
- **contactInfo** (*string/object, optional*) — when available.
- **votePosition** (*string, optional*) — per-vote stance from transcripts.

- `speakerType` (*string, optional*) — `city_commission`, `city_staff`, `public`.
- `nominatedBy` (*string, optional*) — for appointment contexts.

Organization (Organization)

ID field: `orgID` New/Added Attributes:

- `subtype` (*string, optional*) — department/division/category.
- `jurisdiction` (*string, optional*) — e.g., `City of Coral Gables`.
- `relationshipToCity` (*string, optional*) — e.g., `County`, `Vendor`, `District`.
- `address` (*string, optional*) — postal address.

Location (Location)

ID field: `locationID` New/Added Attributes:

- `address` (*string, optional*) — e.g., `405 Biltmore Way, Coral Gables, FL 33134`.
- `coordinates` (*string/object, optional*) — lat/long or GeoJSON.

VoteOutcome (VoteOutcome)

ID field: `outcomeID` New/Added Attributes:

- `status` (*string*) — `passed` / `failed` / etc.
- `yesVotes` (*int*) — count of affirmative votes.
- `noVotes` (*int*) — count of negative votes.
- `abstentions` (*int, optional*) — count of abstentions.
- `voteDetails` (*array, optional*) — structured roster.
- `voteType` (*string, optional*) — e.g., `motion`.
- `result` (*string, optional*) — optional textual result label.
- `voters` (*array, optional*) — roster for the vote.
- `agendaItemID` (*string, optional*) — item the vote pertains to.

Action (Action)

ID field: `actionID` New/Added Attributes:

- `actionType` (*string*) — e.g., `approve`, `move`, `second`.
- `target` (*string, optional*) — target entity ID.
- `timestamp` (*string, optional*) — time marker from transcript.

Contract (Contract)

ID field: `contractID` **New/Added Attributes:**

- `vendor` (*string*) — awarded entity.
- `amount` (*number/string*) — awarded value.
- `purpose` (*string*) — scope/description.
- `duration` (*string*) — term.

Asset (Asset)

ID field: `assetID` **New/Added Attributes:**

- `value` (*number/string*) — value relevant to the policy/agenda.
- `fiscalYear` (*string/int*) — fiscal year.
- `taxingDistrict` (*string, optional*) — e.g., special district.

LegalReference (LegalReference)

ID field: `legalReferenceID` **New/Added Attributes:**

- `citation` (*string*) — e.g., `Section 5.1, Chapter 163, F.S.`
 - `codeName` (*string, optional*) — name of the code/body.
 - `jurisdiction` (*string, optional*) — e.g., `County, State`.
 - `url` (*string/URL, optional*) — public link.
-

Common / Ingestion Attributes

These fields are widely used across entities for identification, lineage, and debugging.

- `id` (*string, optional*) — canonical ID.
- `partitionKey` (*string, optional*) — logical partition (e.g., `cgGraph`).
- `entity_type` (*string, optional*) — ontology class label (e.g., `Document`).
- `_sources` (*array of strings, optional*) — provenance markers (e.g., `ner_*`).
- `_source` (*string, optional*) — single-source marker.
- `_created_at` (*string/ISO datetime, optional*) — creation timestamp.
- `_notes` (*object, optional*) — annotations.
- `metadata` (*object, optional*) — arbitrary, source-native metadata.
- `source_file_name` (*string*) — (Alias: `Source_File_Name`).
- `source_file_path` (*string*) — (Alias: `Source_File_Path`).
- `meetingDate` (*date/string*) — meeting context date.
- `status` (*string*) — lifecycle label.

Notes on Naming & Aliases

- Where an ingestion step produces mixed casing (e.g., `Source_File_Name`), treat the **canonical** form as lower-snake: `source_file_name`. Both are accepted during ingestion; storage should prefer canonical names.
- `documentType` is the **only** place where a domain "type" should be stored for documents. Other entities that need a local category should use `subtype` to avoid colliding with the ontology's `type` label.

Conclusion: Architectural Patterns for Real-World Deployment

From Theory to Practice: Municipal Document Intelligence demonstrates that the architectural limitations and hybrid systems described above are not academic abstractions. They manifest acutely in real-world systems where correctness, completeness, and organizational memory are non-negotiable. Municipal document management represents a perfect example of these challenges: tens of thousands of technical documents with complex interdependencies, legal requirements for complete retrieval, evolving organizational terminology, and critical need for factual accuracy. The case study outlined how I confronted these exact limitations and experimented with various architectural approaches to develop a blueprint for more intelligent document use by combining knowledge graphs and large language models.

The power of augmenting LLMs with knowledge graphs extends far beyond improved document retrieval. Knowledge graphs provide the structural scaffold that transforms LLMs from impressive but unreliable generators into trustworthy components of enterprise systems. While LLMs contribute natural language understanding and generation capabilities, knowledge graphs enforce the constraints, completeness guarantees, and relationship preservation that mission-critical applications require. This architectural pairing creates systems where the linguistic flexibility of LLMs coexists with the deterministic precision of graph structures, each compensating for the other's limitations.

The broader "graph-of-graphs" architecture proposed for city-wide implementation would extend these benefits further. By modeling each city subsystem as its own knowledge graph unified under a master ontology, this approach would enable seamless and reliable interoperability between diverse datasets. The knowledge graph structure would act as a semantic integration layer, standardizing entities and relationships across disparate data sources while preserving their unique contexts. This integration would facilitate advanced analytics pipelines, enable cross-domain machine learning models, power automated decision support systems, and support complex reasoning tasks that require information from multiple domains. While each implementation will require domain-specific adaptations and ongoing maintenance, the architectural patterns and blueprint presented here provide a foundation for building intelligent document systems across diverse domains. This hybrid approach points toward a future where organizations can finally achieve the promise of AI-powered document intelligence: systems that are not just linguistically fluent, but structurally sound, organizationally aware, and operationally reliable.

Acknowledgments

I would like to express my sincere gratitude to Mark, Javier, and Ana at the GIS & IoT Lab at the City of Coral Gables for their invaluable guidance and mentorship throughout this internship. Their expertise, patience, and willingness to entertain my intellectual explorations created an environment where experimental approaches could flourish. The insights and support they provided were instrumental in developing the architectural blueprint presented in this paper.

References:

- 1) Barrasa, J., & Webber, J. (2023). *Building knowledge graphs*. O'Reilly Media.
<https://www.oreilly.com/library/view/building-knowledge-graphs/9781098127091/>
- 2) Lucek, A. (2024). *Knowledge graph or vector database... which is better?* [Video]. YouTube.
https://www.youtube.com/watch?v=6vG_amAshTk
- 3) AI Engineer. (2025). *HybridRAG: A fusion of graph and vector retrieval — Mitesh Patel, NVIDIA* [Video]. YouTube. <https://www.youtube.com/watch?v=-tgQa8Fzf80>
- 4) Alexopoulos, P. (2025, June 24–25). *Knowledge graphs and large language models bootcamp* [Live online course]. O'Reilly Media.